Building a Pipeline for Time Series Data

Working with Time Series Data: Formative Assignment

Group 13

Josue Byiringiro

Arnold Mutara

Celine Shoga

Repository: https://github.com/jbyiringiro/gdp-time-series-pipeline

October 2026

Contents

1. Introduction 3
2. Problem Definition 3
3. Dataset and Justification 5
4. Time-Series Preprocessing, Exploratory Analysis and Modelling 6
    4.1 Loading and Cleaning 7
    4.2 Time Range and Frequency 7
    4.3 Missing Values 7
    4.4 Distribution of Numerical Variables 8
    4.5 Stationarity 9
    4.6 Feature Engineering 10
    4.7 Analytical Questions 10
    4.8 Model Training and Experimental Design 15
    4.9 Summary of the Preprocessing Pipeline 18
5. Database Design and Implementation 19
    5.1 Relational Schema in MySQL 19
    5.2 Document Design in MongoDB 23
    5.3 Queries 25
    5.4 Comparison of the Two Designs 28
6. API for CRUD Operations and Time-Series Queries 29
    6.1 Correctness of the Create and Update Operations 32
    6.2 Demonstration of the CRUD Workflow 33
7. Prediction Script 34
    7.1 Consistency with Task 1 35
    7.2 Example Run 35
    7.3 Error Handling 37
    7.4 Reproducibility 37
8. Conclusion 37
9. Team Contributions 38
References 38
Appendix A. Database Queries and Results 40
    A.1 MySQL 40
    A.2 MongoDB 42

1. Introduction

This report describes the design and implementation of a pipeline for time-series data built on the Global Economy Indicators dataset (1970–2021). The pipeline consists of four components, which correspond to the four tasks of the assignment:

1. Time-series preprocessing, exploratory data analysis and modelling, which define the prediction problem and produce the trained model (Section 4).
2. A relational database in MySQL and a document database in MongoDB, which store the same records (Section 5).
3. An API that provides create, read, update and delete (CRUD) operations and time-series queries for both databases (Section 6).
4. A prediction script that retrieves records through the API, applies the preprocessing developed in Task 1, loads the trained model and produces a GDP estimate (Section 7).

Sections 2 and 3 define the prediction problem and justify the choice of dataset. Section 8 summarises the results and limitations, and Section 9 describes the contribution of each team member. Table 1 shows how data flow between the components.

Table 1. Components of the pipeline.

Stage	Component	Input	Output
1	Notebook Building_a_Pipeline_for_Time_Series_Data.ipynb (Task 1)	CSV file	Cleaned series, features, model experiments
2	task2/mysql_setup.py, task2/mongodb_setup.py (Task 2)	CSV file	MySQL tables and MongoDB collection with 10,512 records each
3	task3/api_demo.py (Task 3)	MySQL and MongoDB	JSON responses over HTTP
4	task4/predict_from_api.py (Task 4)	API responses and the saved model	GDP estimate for a selected year

2. Problem Definition

The objective is to estimate the annual Gross Domestic Product (GDP) of the United States, in current US dollars, for a given year t. For each year the model receives the GDP history up to year $t-1$, summarised by the previous year's GDP (GDP_Lag1) and the mean GDP of the five previous years (GDP_5Year_MA), together with two indicators for year t: population and final consumption expenditure. The output is a single value, the GDP of year t. Because two of the inputs refer to the same year as the target, the task is a nowcast, that is, an estimate of the current year's GDP from related indicators, rather than a forecast several years ahead.

The use of same-year inputs presupposes that these values are known before the GDP figure itself. The estimate for year t is intended to be produced at the end of that year, before the official annual figure exists. At that point both inputs are largely known from higher-frequency statistics. In the

United States, the Bureau of Economic Analysis publishes personal consumption expenditures every month, about a month after the month ends, and government consumption with the quarterly GDP estimates, while the Census Bureau publishes its estimate of the population on 1 July of the year in December (U.S. Bureau of Economic Analysis, n.d.-b; U.S. Census Bureau, n.d.). The first estimate of annual GDP follows at the end of January of year t+1 (U.S. Bureau of Economic Analysis, n.d.-a), and the UN series used in this project appears later still, because it is compiled from the national figures. In practice, the full-year consumption value would therefore be an estimate based on the first ten or eleven months of the year, whereas the evaluation in this report uses the final, revised annual values. The reported test errors are consequently somewhat lower than those that a real-time estimate would achieve.

Table 2. Specification of the prediction problem.

Item	**Definition**
Unit of analysis	United States, one observation per year
Target	Gross Domestic Product (GDP), current US$
Type of task	Regression on a time series, one year at a time
Training period	1975–2011 (first 80% of the usable years, in time order)
Validation	Three time-ordered folds within the training period (validation years 1985–2011), used to select the model, its hyperparameters and its features
Test period	2012–2021 (last 20%, including the 2020 COVID-19 shock), used once, for the final evaluation, in the revised procedure (Section 4.8)
Evaluation metrics	RMSE (US$), MAPE (%), R^2
Success criterion	Lower test error than the naive forecast in which GDP for year t equals GDP for year $t-1$

GDP is the principal measure of the size of an economy. Governments express budgets, debt and deficit targets as a share of GDP, central banks use it to assess whether the economy is expanding or contracting, and firms use it to estimate the size of markets. The official annual figure is published only after the year has ended and is revised subsequently, so a transparent estimate derived from indicators that are already available has practical value for planning. The United States was selected because it is the largest economy in the dataset and has a complete series (Section 3).

The problem is treated as a time-series problem because GDP in one year builds on GDP in the preceding year, so the order of the observations carries information. Three consequences follow: lagged and moving-average features are constructed; the data are divided into training and test sets by time rather than at random; and the model, its hyperparameters and its features are selected on time-ordered validation folds within the training period, so that the model is never trained on observations from the future and, in the revised procedure, the test period is used only once, for the final evaluation (Hyndman & Athanasopoulos, 2021).

3. Dataset and Justification

The dataset is Global Economy Indicators, published on Kaggle (prasad22, n.d.) and compiled from the National Accounts Main Aggregates Database of the United Nations Statistics Division (United Nations Statistics Division, n.d.). The database is produced in cooperation with national statistical offices and follows a single accounting standard, the System of National Accounts (European Commission et al., 2009). The file contains 10,512 rows, one per country and year, for 220 countries between 1970 and 2021. Its 26 columns cover GDP, Gross National Income (GNI), population, exchange rates, the expenditure components of GDP (consumption, investment, exports and imports) and value added by economic sector.

Table 3. Assignment requirements and how the dataset meets them.

Requirement	**Evidence in the dataset**
Clear timestamp or date column	Year (1970–2021), one record per country per year
Meaningful prediction target	Gross Domestic Product (GDP)
Multiple measurable variables over time	22 numeric indicators recorded every year

The dataset was chosen for five reasons.

1. The predictors have a known relationship with the target. GDP measured by expenditure is the sum of consumption, investment and net exports, GDP = C + I + (X − M). The dataset contains each of these components, and for the United States they sum to the reported GDP within 0.53% in every year. The predictive value of the inputs is therefore grounded in national accounting rather than in incidental correlation.
2. All values come from one official source and follow one method, so years and countries are comparable. This is essential for a time series, since a change in the way data are collected would otherwise appear as a change in the economy.
3. The period is long enough to include economic shocks. The 52 years cover several business cycles, including the 2008–09 financial crisis and the 2020 pandemic, which produced the only two annual declines in US GDP. The test period (2012–2021) contains the 2020 shock, so the evaluation is not limited to stable years.
4. The data are nearly complete. Only 0.87% of all values are missing, and 78% of these are in a single column (Changes in inventories) that is not used by the model.
5. The structure suits the entire pipeline. The country-by-year layout maps directly onto the relational schema of Task 2 and onto one MongoDB document per country and year, and annual records can be served by date range through the API of Task 3.

The United States was selected for the following reasons, each of which was verified in the notebook (Figure 1):

- It is the largest economy in the dataset, with a GDP of US$23.3 trillion in 2021 compared with US$17.7 trillion for China, and it accounted for between 21% and 32% of world GDP in every year since 1970 (31.4% in 1970 and 24.3% in 2021).
- Its series is complete: all 52 years are present and none of the modelled columns has a missing value.

- Its GDP is free of currency distortion. Values are recorded in US dollars and the US exchange rate equals 1.0 in every year, whereas for other countries the dollar value of GDP combines real economic change with exchange-rate movements.
- The expenditure identity holds closely: consumption, investment, exports and imports reproduce the reported GDP within 0.53% in every year, and consumption alone averages 80.2% of GDP.

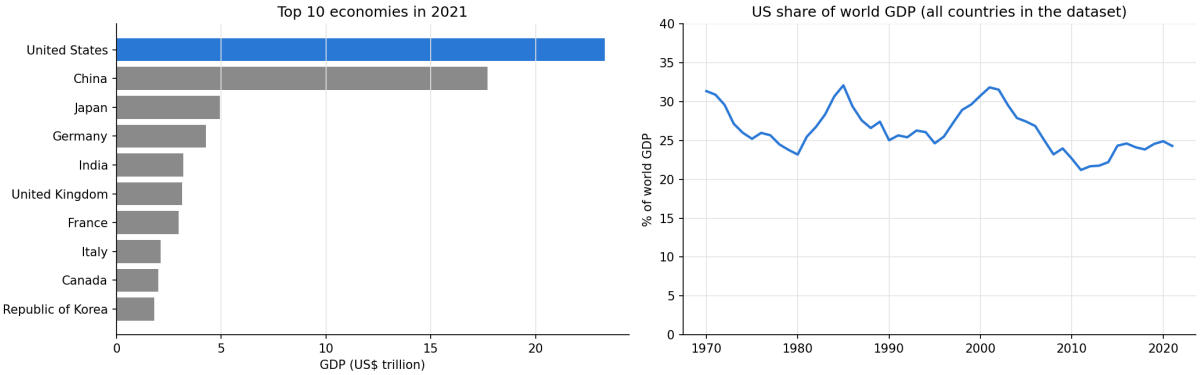


Figure 1. Left: the ten largest economies in 2021, with the United States highlighted. Right: the US share of world GDP in the dataset, 1970–2021. The share moves with the value of the dollar, because the GDP of other countries is converted into dollars.

The dataset also has limitations, which are summarised in Table 4 together with the measures taken to address them.

Table 4. Limitations of the dataset.

Limitation	**Effect**	**Mitigation**
Annual data: 52 observations for one country (47 after feature construction)	Little data for learning; complex models overfit	Few features, simple models, small tuning grids and a strict time-based split
Values in current (nominal) dollars	Growth includes inflation as well as higher output	Nominal GDP is predicted and growth rates are reported as nominal
US values rounded to three significant figures	Changes below about 0.5–1% are partly due to rounding	Differences in error below about 1% are not treated as meaningful

4. Time-Series Preprocessing, Exploratory Analysis and Modelling

This section describes Task 1. All steps are implemented in the notebook Building_a_Pipeline_for_Time_Series_Data.ipynb, which also saves every figure in this section to report/figures/.

4.1 Loading and Cleaning

The CSV file was loaded and the surrounding spaces were removed from the column names and from the text columns Country and Currency. In the raw file every header and text value is padded with spaces (for example, ' Afghanistan '), so without this step a filter on the country name returns no rows and column look-ups by name fail. After cleaning, the table has 26 columns: two identifiers (CountryID and Country), the time column Year, the text column Currency and 22 numeric indicators.

4.2 Time Range and Frequency

The first and last years, the interval between consecutive records of each country, duplicate country-year rows and the number of countries reporting in each year were examined (Figure 2). These checks are necessary because lag and moving-average features assume that each row represents exactly one year; if years were missing or duplicated, a shift of one row would not correspond to the previous year.

The data cover the period 1970 to 2021 (52 years) at annual frequency. Every interval between consecutive records is exactly one year and there are no duplicate country-year rows, so no resampling is required. Of the 220 countries, 179 have all 52 years. The number of reporting countries rises from 187 to 216 in 1990, when the successor states of the USSR, Yugoslavia and Czechoslovakia first appear, and falls to 211 in 1991, when the former states cease to report. In 1990 both the former states and their successors are present, so world totals for that year count these economies twice. This does not affect the US series used for modelling.

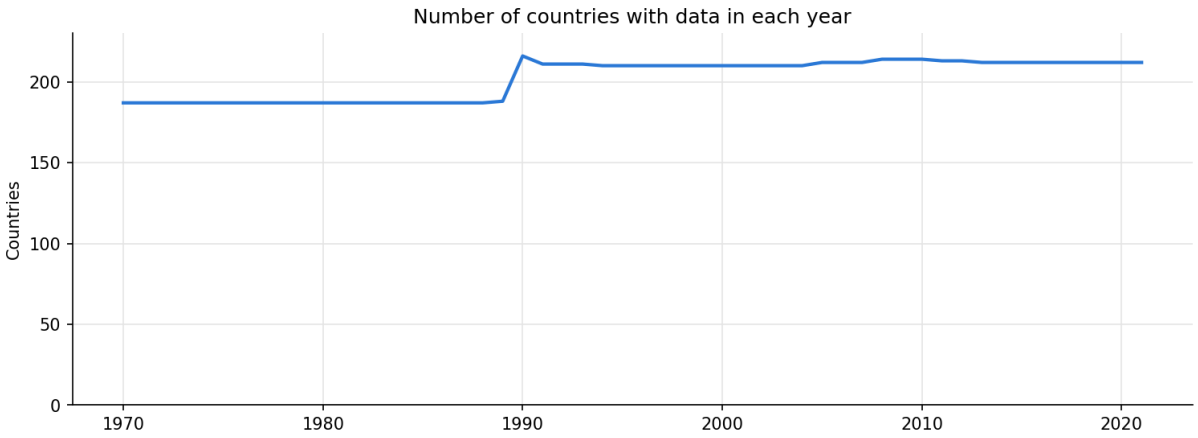


Figure 2. Number of countries with data in each year. The step in 1990 reflects the dissolution of the USSR, Yugoslavia and Czechoslovakia.

4.3 Missing Values

In total, 2,374 values (0.87% of all cells) are missing, and 78% of them are in Changes in inventories, where 17.5% of the rows are empty. Every other column is at least 98.8% complete (Figure 3). The US series contains only two missing values, both in Changes in inventories, for 1970 and 1971.

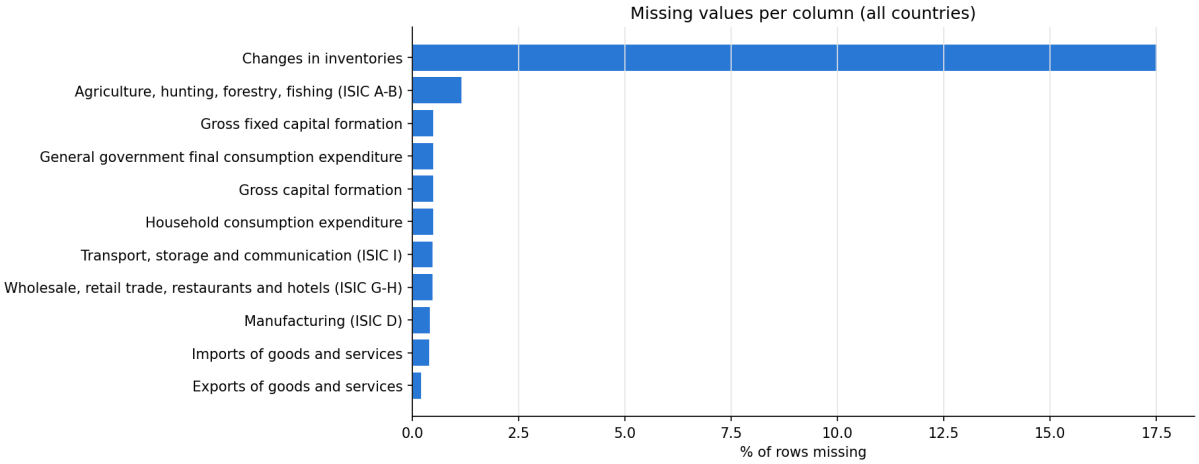


Figure 3. Percentage of missing values per column across all countries. Only columns with at least one missing value are shown.

Missing values were treated as follows:

1. Gaps between two known years were filled by linear interpolation. Economic indicators change gradually from year to year, so a straight line between neighbouring years preserves the trend. Mean imputation would be inappropriate for a trending series: the mean US GDP (US$9.3 trillion) is almost nine times the 1970 value, so a mean-filled value in the 1970s would create a large artificial spike.
2. Gaps at the beginning or end of a series were filled from the nearest known year (backward or forward fill), because interpolation requires a known value on both sides. The two US gaps (1970 and 1971) are of this kind and take the 1972 value.
3. Rows were not removed. Deleting 1970 and 1971 would also discard GDP, population and consumption for those years, although these columns are complete, and would interrupt the continuous sequence of years on which the lag and moving-average features depend.

After this step the US series has no missing values. Since Changes in inventories is not a model input, the filled values do not affect the model.

4.4 Distribution of Numerical Variables

Summary statistics and skewness were computed for the target and the main explanatory variables (Table 5), and their distributions were plotted (Figure 4).

Table 5. Summary statistics for the United States, 1970–2021.

Statistic	**GDP (US$ trillion)**	**Population (million)**	**Consumption (US$ trillion)**	**GNI per capita (US$)**
Mean	9.32	268.1	7.60	32,119
Median	7.86	267.3	6.26	29,213
Standard deviation	6.50	43.3	5.42	18,944
Minimum (1970)	1.07	200.3	0.84	5,363

Statistic	GDP (US$ trillion)	Population (million)	Consumption (US$ trillion)	GNI per capita (US$)
Maximum (2021)	23.30	337.0	19.30	70,081
Skewness	0.48	0.05	0.48	0.28

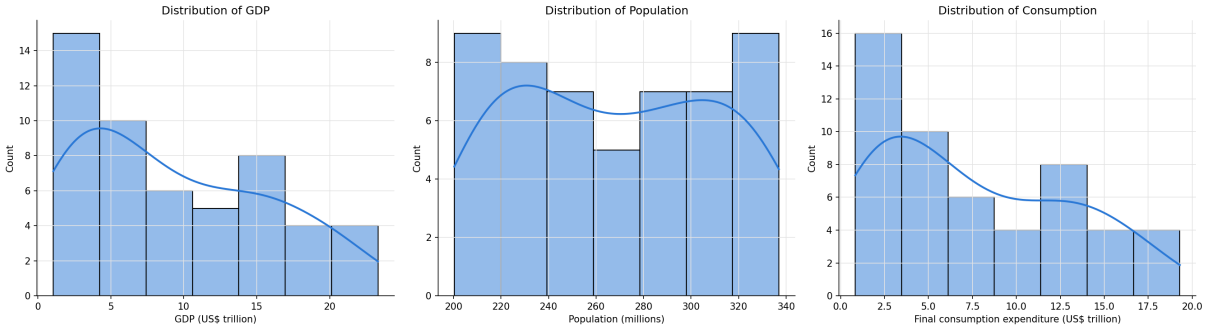


Figure 4. Distributions of GDP, population and final consumption expenditure for the United States, 1970–2021.

GDP is right-skewed (skewness 0.48), with the mean (US$9.32 trillion) above the median (US$7.86 trillion). Consumption has the same shape because it represents a stable share of about 80% of GDP. Population is almost symmetric (skewness 0.05) because it grows by a similar number of people each year, and GNI per capita shows a mild right skew (0.28). For a time series, these distributions describe when values occurred rather than random variation: because GDP grows by a percentage each year, it spends many years at low values and few at high values, which produces the right skew. The mean and variance of GDP therefore depend on the period considered, a property that was tested formally in the next step.

4.5 Stationarity

The Augmented Dickey-Fuller (ADF) test (Dickey & Fuller, 1979), as implemented in statsmodels (Seabold & Perktold, 2010), was applied to the GDP level and to its annual growth rate. The null hypothesis of the test is that the series has a unit root, that is, that it is non-stationary and its mean and variance change over time. A p-value below 0.05 rejects the unit root; a higher p-value means that the test fails to reject it, which is consistent with non-stationarity but does not prove it.

Table 6. Results of the Augmented Dickey-Fuller test.

Series	ADF p-value	Result
GDP level	0.997	Failed to reject a unit root
GDP annual growth rate (%)	0.211	Failed to reject a unit root

For the GDP level the test fails to reject a unit root ($p = 0.997$), which is consistent with its strong trend. It also fails to reject a unit root for the growth rate ($p = 0.211$), consistent with nominal growth having been considerably higher during the high-inflation 1970s (10.5% per year in 1971–1981) than after 2000 (about 4%). With only 51–52 annual observations the test has limited power, so these results do not prove that the series are non-stationary; together with the visible trend, however, they indicate that GDP cannot be treated as stationary. These results shaped three later decisions. First, the

test years contain GDP values above any value in the training years, so the model must be able to extrapolate, which favours linear models over tree-based models. Second, two trending series always appear correlated, so correlations are examined on growth rates as well as on levels (Questions 3 and 4). Third, the features describe the recent history of the series, so that the model works relative to the current level of GDP.

4.6 Feature Engineering

Table 7 lists the features constructed from the GDP series.

Table 7. Engineered time-series features.

Feature	Definition	Purpose
GDP_Lag1	GDP of year $t-1$ (shift(1))	The strongest single predictor (Question 3)
GDP_5Year_MA	Mean GDP of years $t-5$ to $t-1$ (shift(1) followed by rolling(5))	A stable summary of recent history that smooths single-year shocks (Question 2)
GDP_Growth	Year-on-year change in per cent (pct_change())	Removes the trend; used throughout the analysis

To prevent data leakage, the GDP series is shifted by one year before the rolling mean is computed. Without the shift, the average for year t would include the GDP of year t, which is the value being predicted. A five-year window is long enough to smooth a single exceptional year and short enough to retain 47 of the 52 years for modelling, since the first complete moving average is available for 1975. The rows for 1970–1974 are removed immediately before modelling, so that the exploratory analysis can use the full period.

4.7 Analytical Questions

Five analytical questions were investigated (Table 8). Question 2 is based on moving averages and Question 3 on lagged values.

Table 8. Analytical questions and techniques.

No.	Question	Technique
1	Does GDP show a consistent increasing trend over the decades?	Trend and growth rates
2	How does the five-year moving average smooth economic volatility?	Moving averages
3	Is the previous year's GDP a strong predictor of the current year's GDP?	Lagged values and autocorrelation
4	Which variables correlate most strongly with GDP growth?	Correlation of levels and of growth rates
5	How has GDP per person evolved relative to population growth?	Indexing to a common base year

Question 1: Does GDP show a consistent increasing trend over the decades?

This question examines whether the series is stable or trending and whether the trend is interrupted, since a trending series requires a model that can predict values above those observed during training.

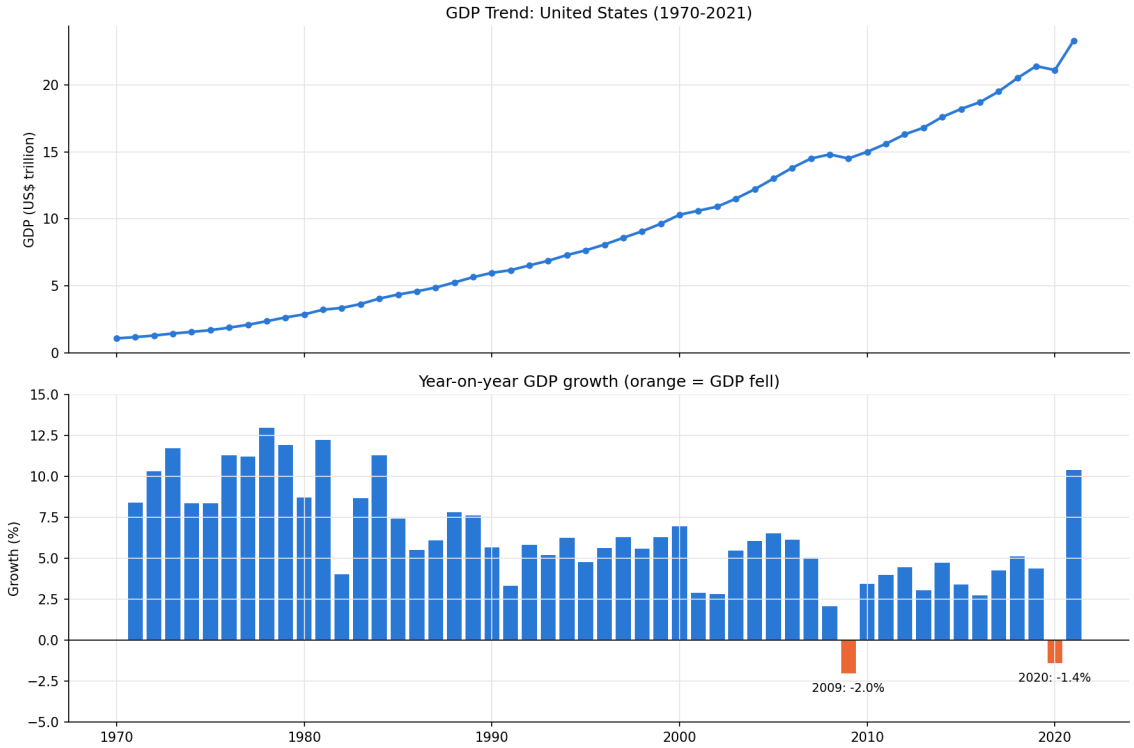


Figure 5. Top: US GDP, 1970–2021. Bottom: year-on-year growth; the two years in which GDP declined are marked.

GDP rose from US$1.07 trillion in 1970 to US$23.3 trillion in 2021, an increase by a factor of about 22, or 6.2% per year on average. The increase was steady: GDP grew in 49 of the 51 annual changes. It declined only in 2009 (−2.0%), following the 2008 financial crisis, and in 2020 (−1.4%), during the COVID-19 pandemic, and both declines were recovered within one to two years. The growth rate itself was not constant: nominal growth averaged 10.5% per year in 1971–1981, a period of high inflation, and about 4% after 2000. Because the test years (2012–2021) contain GDP values above any value in the training years, the model must be able to extrapolate.

Question 2: How does the five-year moving average smooth economic volatility?

This question examines how a rolling average removes year-to-year variation, and how far the moving-average feature lies from the value to be predicted.

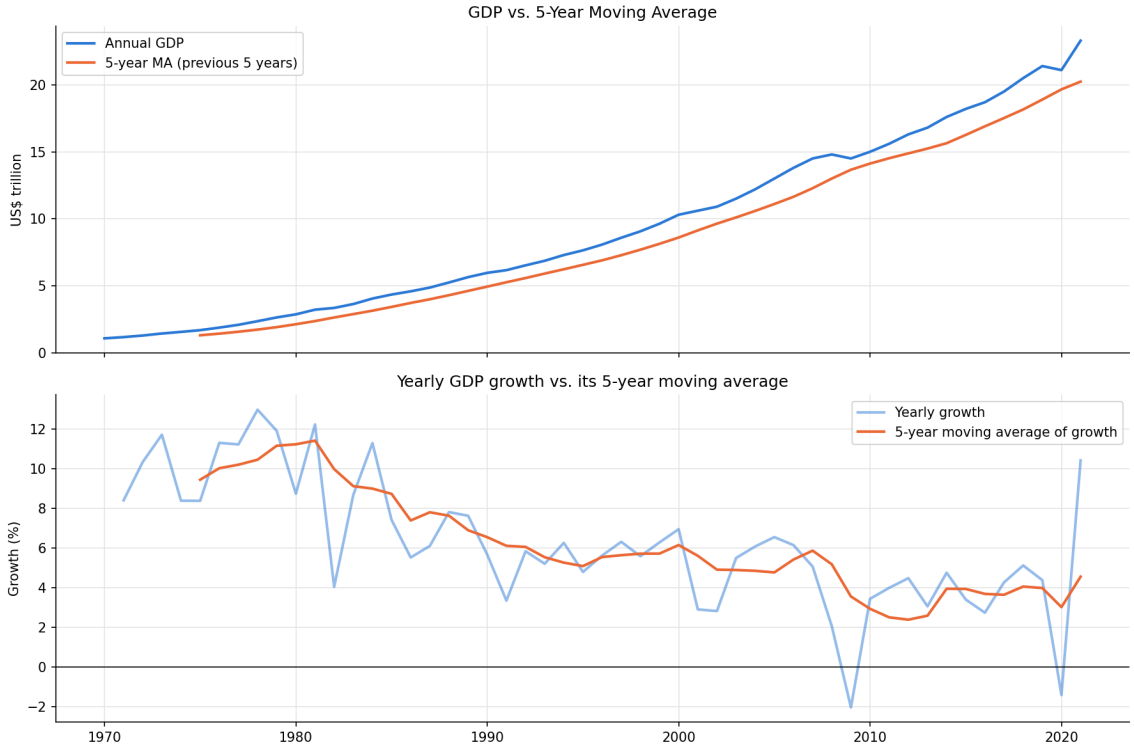


Figure 6. Top: annual GDP and the moving-average feature (mean of the five previous years). Bottom: annual growth and its five-year moving average.

On the level chart, the moving average forms a smooth line that continues to rise through the declines of 2009 and 2020. It lies below GDP by 19% on average, because on a rising trend the mean of the five previous years is always lower than the current value. The moving average alone would therefore under-predict, and the model must learn how far above the recent average GDP will lie. The growth chart shows the smoothing more clearly. Annual growth ranges from −2.0% to 13.0% (standard deviation 3.3 percentage points), whereas its five-year average ranges from 2.4% to 11.4% (standard deviation 2.5 points) and reveals long-run phases rather than individual shocks: about 10% in the 1970s, declining through the 1980s and 1990s to about 4% after 2000. The five-year moving average is therefore a stable summary of recent economic conditions, and it is used as a model feature alongside the previous year's GDP.

Question 3: Is the previous year's GDP a strong predictor of the current year's GDP?

This question tests for autocorrelation, the correlation of a series with a lagged copy of itself, for lags of one to ten years. Both the GDP level and the annual growth rate are examined, because a shared trend alone can make levels appear correlated.

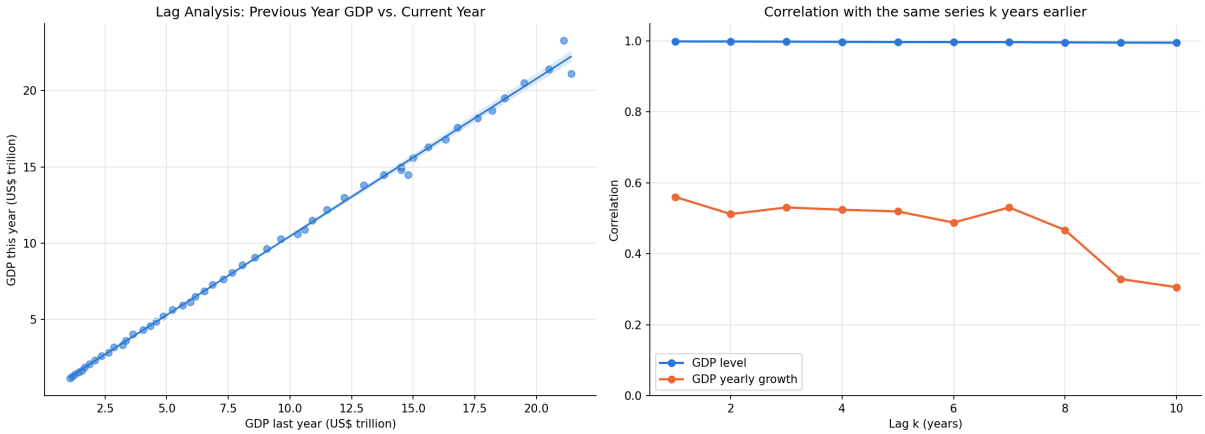


Figure 7. Left: GDP in the current year against GDP in the previous year. Right: correlation of GDP (level and growth rate) with the same series k years earlier.

Table 9. Autocorrelation of GDP at selected lags.

Lag (years)	**1**	**2**	**3**	**5**	**8**	**10**
GDP level	0.999	0.999	0.998	0.997	0.996	0.995
GDP growth rate	0.561	0.512	0.531	0.519	0.467	0.306

In levels, the correlation between the current and the previous year's GDP is 0.999, and the points lie almost exactly on a straight line. However, the level correlation remains above 0.995 even at a lag of ten years, which shows that much of it reflects the common trend rather than genuine predictive information. Growth rates remove the trend and give a more informative result: 0.56 at lag 1, about 0.5 up to lag 8 and 0.31 at lag 10. Strong years tend to be followed by strong years, partly because of the long phases identified in Question 2. GDP_Lag1 is therefore the most important feature, and it also defines the benchmark that any model must outperform: the naive forecast in which GDP in the current year equals GDP in the previous year.

Question 4: Which variables correlate most strongly with GDP growth?

This question identifies the explanatory variables that move with GDP. Because all of these series trend upwards, the correlations of their levels are compared with the correlations of their annual growth rates. A variable that correlates only in levels shares the trend but does not move with GDP from year to year.

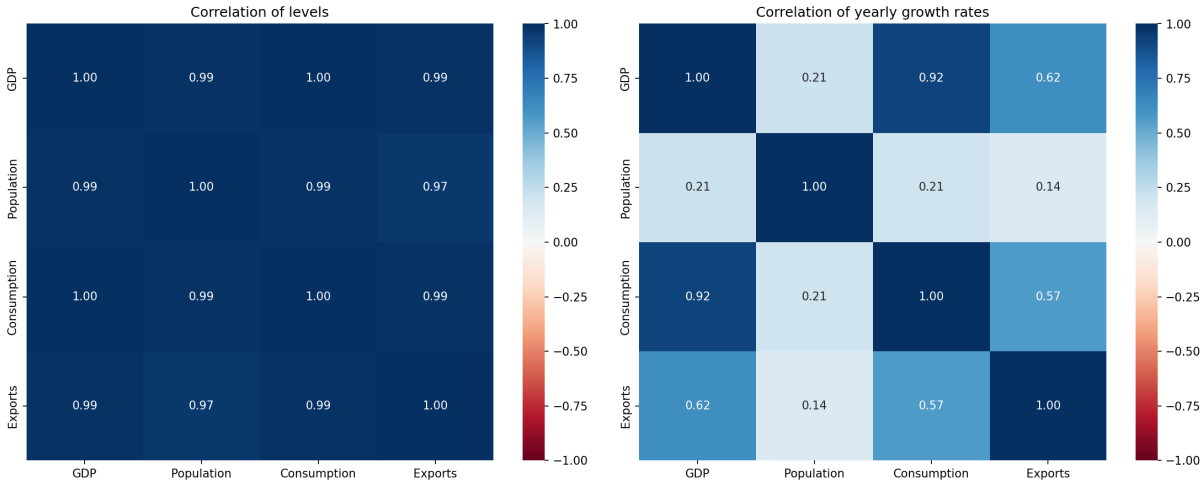


Figure 8. Correlation matrices of GDP, population, final consumption expenditure and exports. Left: levels. Right: annual growth rates.

Table 10. Correlation with GDP in levels and in growth rates.

Variable	**Levels**	**Growth rates**
Final consumption expenditure	1.00	0.92
Exports of goods and services	0.99	0.62
Population	0.99	0.21

In levels, every variable correlates with GDP at 0.99 or above, which would suggest that population is almost as informative as consumption. Growth rates distinguish between them. Consumption (0.92) grows faster when GDP grows faster, as expected for a component that accounts for about 80% of GDP. Exports (0.62) show a moderate relationship, consistent with trade being a smaller and more volatile part of GDP. Population (0.21) is only weakly related: it increases by a similar amount each year regardless of economic conditions, and its correlation of 0.99 in levels arises from the shared upward trend. Consumption is therefore retained as the strongest explanatory variable. Population is retained as a slow-moving measure of the size of the economy, although it is expected to contribute little; Section 4.8 tests this expectation.

Question 5: How has GDP per person evolved relative to population growth?

This question examines whether GDP grew mainly because the population increased or because output per person increased. Since GDP equals population multiplied by GDP per person (GDP divided by population), the growth of these two components accounts exactly for the growth of GDP. GDP, GDP per person and population are indexed to 1970 = 100 and plotted on a single logarithmic axis, so that the comparison does not depend on the scaling of separate axes.

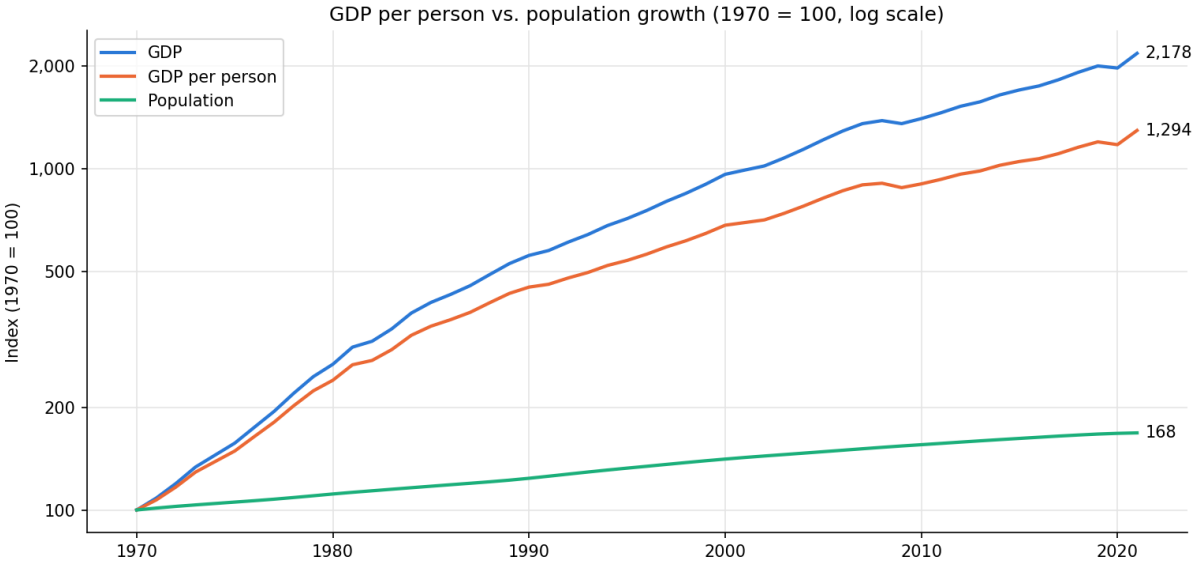


Figure 9. GDP, GDP per person and population indexed to 1970 = 100 (logarithmic scale). The labels show the 2021 index values.

Between 1970 and 2021 the population grew by a factor of 1.68 (from 200 to 337 million), GDP per person by a factor of 12.9 (from US$5,341 to US$69,140) and GDP by a factor of 21.8. Because GDP equals population multiplied by GDP per person, the two factors multiply exactly to the factor for GDP; with the rounded values, $1.68 \times 12.94 \approx 21.78$. In logarithmic terms, population growth accounts for about 17% of GDP growth and higher GDP per person for the remaining 83%. GDP per person is used instead of the GNI per capita reported in the dataset, because population multiplied by GNI per capita yields GNI rather than GDP; for the United States the two measures are nevertheless close (GNI per capita grew by a factor of 13.1), since GNI differs from GDP only by net income from abroad. Since the figures are in current dollars, part of the rise in GDP per person reflects inflation, but population growth remains far too slow to explain the growth in GDP. This agrees with the weak growth-rate correlation found in Question 4: year-to-year changes in GDP originate in income and spending, so recent GDP and consumption carry most of the predictive information.

4.8 Model Training and Experimental Design

The candidate features are those suggested by the analysis: GDP_Lag1 (Question 3), GDP_5Year_MA (Question 2), final consumption expenditure (Question 4) and population. Rather than fixing the feature set in advance, five sets are compared: GDP_Lag1 alone; GDP_Lag1 with GDP_5Year_MA; GDP_Lag1 with consumption; GDP_Lag1 with GDP_5Year_MA and consumption; and all four features. The models and the selection procedure were implemented with scikit-learn (Pedregosa et al., 2011). The experimental design is as follows:

1. The rows for 1970–1974, which lack a complete moving average, are removed, leaving 47 years.
2. The data are divided chronologically: 1975–2011 (37 years) for training and 2012–2021 (10 years) for testing. A random split would allow the model to be trained on later years and tested on earlier ones, which would leak future information. In the revised procedure, the test period is set aside and used only once; an earlier version of the analysis had examined it, as discussed below.

3. The model, its hyperparameters and its features are selected within the training period. TimeSeriesSplit with three splits forms three folds, each of which is trained on the years that precede its validation block (1985–1993, 1994–2002 and 2003–2011). Every combination of model, hyperparameter setting and feature set is scored by its mean root mean squared error (RMSE) over the three folds, and the combination with the lowest validation RMSE is selected.
4. The selected combination is refitted on the full training period and evaluated once on the test period.
5. The naive forecast, in which GDP for year t equals GDP for year $t-1$, serves as the baseline (Hyndman & Athanasopoulos, 2021).
6. Performance is measured by RMSE, which expresses the typical error in US dollars and penalises large errors; by the mean absolute percentage error (MAPE); and by the coefficient of determination (R^2).

Table 11. Experiments and hyperparameter search. Each model is evaluated with each of the five feature sets.

No.	Model	Rationale	Hyperparameters searched	Setting selected on validation
0	Naive baseline	Benchmark from Question 3	None	None
1	Linear Regression	Able to extrapolate a trend (Question 1)	None (closed-form solution)	None
2	Decision Tree	Tests non-linear splitting rules	max_depth: 3, 5, 10, None; min_samples_split: 2, 5, 10	max_depth 5, min_samples_split 2
3	Random Forest	Averages many trees to reduce variance	n_estimators: 50, 100; max_depth: None, 5, 10	50 trees, max_depth None

Table 12. Validation results within the training period (mean RMSE over the three folds).

Model	Feature set	Validation RMSE (US$ billion)
Linear Regression	All four features	153
Linear Regression	GDP_Lag1	212
Linear Regression	GDP_Lag1, consumption	237
Linear Regression	GDP_Lag1, GDP_5Year_MA, consumption	238
Linear Regression	GDP_Lag1, GDP_5Year_MA	242
Naive baseline	GDP_Lag1	466
Decision Tree (tuned)	Each of the five sets	2,523
Random Forest (tuned)	Each of the five sets	2,667

Linear Regression with all four features achieves the lowest validation RMSE (US$153 billion, compared with US$466 billion for the naive baseline) and is therefore selected. Every Linear Regression outperforms the baseline in validation. GDP_Lag1 alone performs better than its combinations with the moving average or with consumption, but the complete set, which adds

population, is the best by a clear margin. This result revises the expectation formed in Question 4 that population would contribute little: its year-to-year growth is only weakly related to that of GDP, but its level rises smoothly and probably provides the regression with a stable trend term that the other features lack.

The tree-based models perform far worse and obtain identical scores with every feature set. Both results have the same cause. A decision tree can only output averages of training values and therefore cannot extrapolate, while each validation block contains higher GDP values than any year in its training data. In the last fold, the highest GDP in the training years (1975–2002) is US$10.9 trillion and the Random Forest never predicts more than US$10.75 trillion, whereas actual GDP in 2003–2011 lies between US$11.5 and 15.6 trillion. Because every candidate feature rises almost steadily over time, the trees divide the years at the same points regardless of the features used, so their predictions do not depend on the feature set. Since tuning cannot remove this limitation, the tree-based models are rejected before the test period is used.

Table 13. Results of the selected model on the test years (2012–2021), evaluated once in the revised procedure.

Model	**RMSE (US$ billion)**	**MAPE (%)**	**R^2**
Naive baseline	968	4.18	0.79
Linear Regression (selected)	889	4.17	0.82

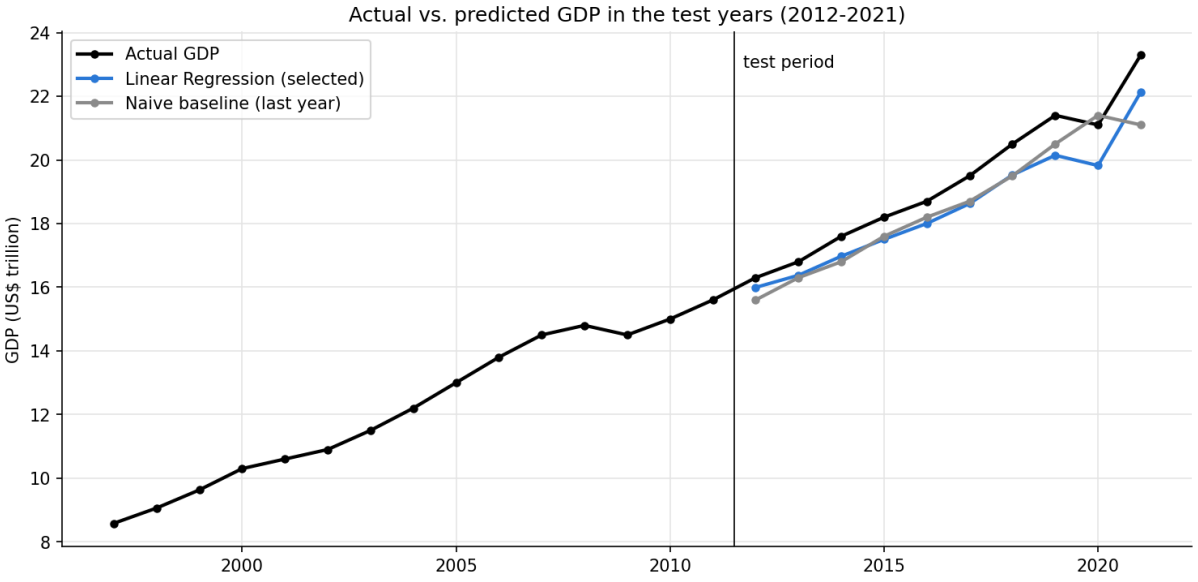


Figure 10. Actual GDP, 1997–2021, and the predictions of the selected Linear Regression model and the naive baseline for the test years.

Table 14. Prediction error by test year (% of actual GDP).

Model	**2012**	**2013**	**2014**	**2015**	**2016**	**2017**	**2018**	**2019**	**2020**	**2021**
Linear Regression	−1.9	−2.5	−3.6	−3.8	−3.7	−4.5	−4.7	−5.9	−6.1	−5.0
Naive baseline	−4.3	−3.0	−4.5	−3.3	−2.7	−4.1	−4.9	−4.2	+1.4	−9.4

Evaluated on the test period after the selection, the selected Linear Regression outperforms the

baseline and thereby meets the success criterion, with an RMSE 8% lower (US$889 billion compared with US$968 billion) and an R^2 of 0.82 compared with 0.79. Because it can extrapolate, it follows GDP above the range observed in training. The margin is modest, and the MAPE of the two is almost identical (about 4.2%), which confirms the finding of Question 3 that the previous year's GDP already contains most of the relevant information.

The test RMSE is considerably higher than the validation RMSE (US$889 billion compared with US$153 billion). Part of the difference is a matter of scale: GDP in 2012–2021 is far higher than in the validation years, so an error of the same percentage corresponds to a larger amount in dollars. The remainder reflects the growing distance between the test years and the training period, discussed below. In the revised procedure, the model, its features and the hyperparameters of the tree-based models are all selected on the validation folds, and the test period is used only after this selection. The test period is nevertheless not fully independent: an earlier version of this analysis compared all models directly on 2012–2021, so these years were examined during development. The validation results lead to the same choice of model and features, but the test results should be regarded as a somewhat optimistic estimate of performance on new data; a fully independent evaluation would require observations for years after 2021.

Linear Regression under-predicts every test year, and the error widens from −1.9% in 2012 to −6.1% in 2020. This indicates that the relationship estimated from 1975–2011 becomes less accurate the further the model predicts beyond its training period. During the 2020–21 shock the model remains within about 6% of the actual value, because the same-year consumption figure falls and recovers with GDP. The naive baseline is 3–5% too low in ordinary years, over-predicts 2020 (+1.4%) and records the largest single error in 2021 (−9.4%), when GDP rebounded by 10%. Linear Regression obtains its lower RMSE mainly by avoiding this large error. Retraining the model each year as new data become available (walk-forward validation) would limit how far it extrapolates and is expected to reduce the growing bias.

Consumption and population enter the model with their values for the same year as the GDP being estimated, which is why the problem is defined as a nowcast; Section 2 explains when these values become available. A strict one-year-ahead forecast would use the previous year's values of these indicators instead. Since consumption accounts for about 80% of GDP, such a variant would be expected to be less accurate and the naive baseline more difficult to outperform.

4.9 Summary of the Preprocessing Pipeline

Table 15 summarises the preprocessing steps. The same steps, with the exception of the exploratory analysis in step 5, are reused by the prediction script in Task 4 (Section 7).

Table 15. Preprocessing steps and their rationale.

Step	Operation	Rationale
1	Remove spaces from column names and text values	Padded names cause filters and look-ups to fail
2	Select the United States, sort by year, use Year as index	Largest complete economy without exchange-rate effects; time order is required for lags

Step	Operation	Rationale
3	Confirm one record per year without gaps or duplicates	Lag features assume that one row corresponds to one year
4	Linear interpolation inside the series, backward or forward fill at the edges	Preserves the trend; mean or zero imputation would create artificial spikes
5	Summary statistics, histograms and the ADF test	Revealed a strong trend, which guided the choice of features and models
6	GDP_Lag1: GDP of the previous year	Strongest single predictor (Question 3)
7	GDP_5Year_MA: mean of the five previous years, shifted before rolling	Smooths shocks (Question 2) without leaking the current year's GDP
8	Remove 1970–1974	Rows without a full five-year history cannot be used for training
9	Train on 1975–2011, test on 2012–2021; select the model, hyperparameters and features on time-ordered validation folds of the training period	The model must never learn from the future, and the test result must not influence the choice of model

5. Database Design and Implementation

This section describes Task 2. The records are stored in two databases: a relational database in MySQL and a document database in MongoDB. Both are created and loaded from the CSV file by Python scripts (task2/mysql_setup.py and task2/mongodb_setup.py), which also execute five queries each.

5.1 Relational Schema in MySQL

The relational design separates the data into three tables (Table 16). Country attributes that do not change over time, the name and the currency, are stored once in countries rather than repeated in each of the 10,512 yearly rows, which removes redundancy and keeps names consistent. Exchange rates are held in a separate table, exchange_rates, because they are monetary measures that describe the currency rather than the output of the economy, and most analytical queries do not require them. The yearly macroeconomic indicators, including GDP, form the central time-series table, economic_indicators.

Table 16. Tables of the relational schema.

Table	Primary key	Foreign key	Content	Rows
countries	country_id	None	Name and currency of each country	220
exchange_rates	id	country_id references countries	AMA and IMF-based exchange rates per country and year	10,512
economic_indicators	id	country_id references countries	20 yearly indicators, including GDP, population and consumption	10,512

Each relationship is one-to-many: one country has many yearly exchange-rate records and many

yearly indicator records (Figure 11). A unique constraint on (country_id, year) in both dependent tables guarantees one record per country and year, a property on which the time-series features rely. Indexes on year and on (country_id, year) support the most frequent access patterns of the API: retrieving the records of one country in year order, the latest year and a range of years. Listing 1 shows the data definition statements.

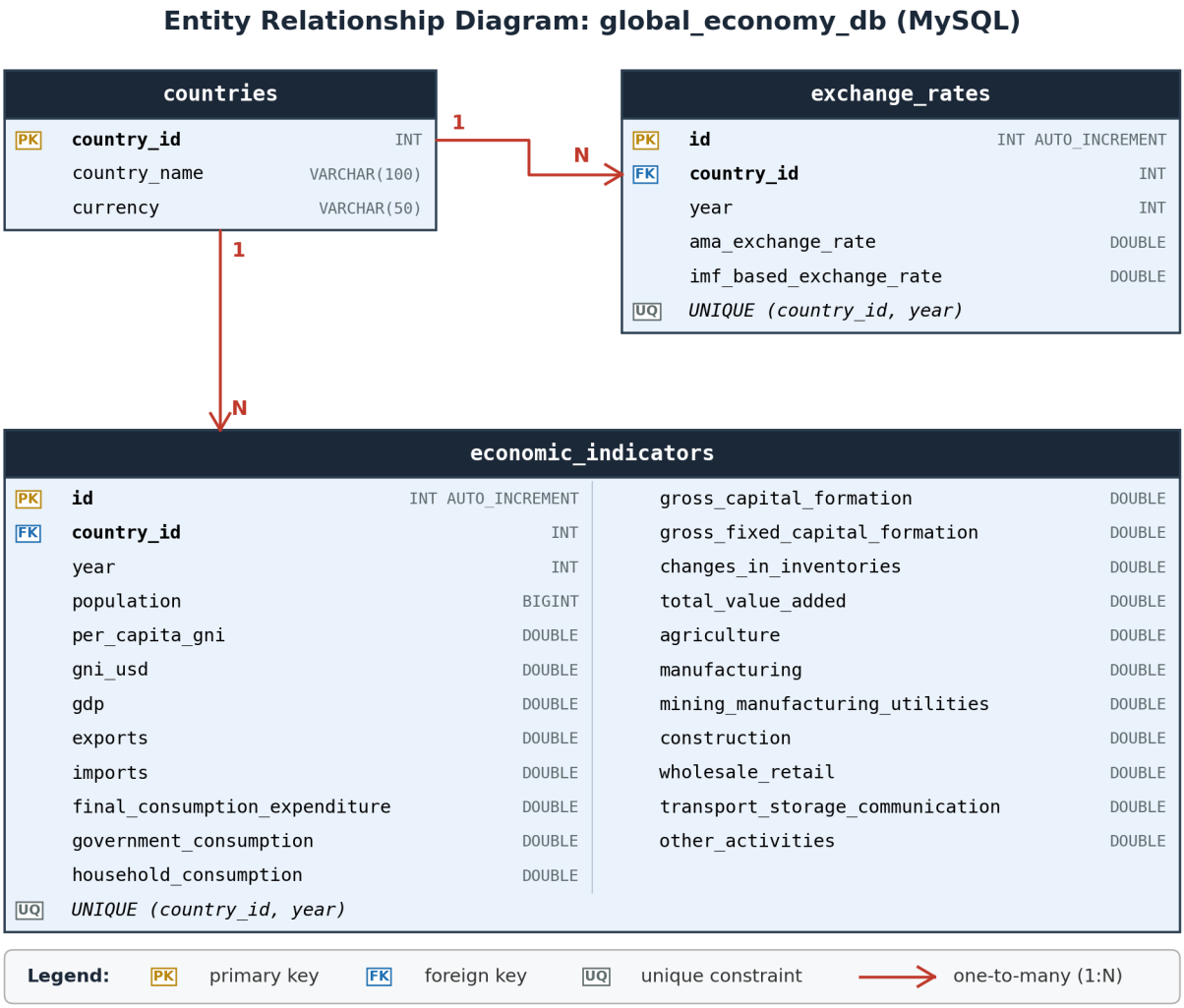


Figure 11. Entity relationship diagram of the MySQL database. Each country has many exchange-rate records and many economic-indicator records; the unique constraints on (country_id, year) are marked UQ.

Listing 1. Data definition statements of the relational schema (abridged; full version in task2/sql_schema.sql).

```
CREATE TABLE countries (
    country_id   INT PRIMARY KEY,
    country_name VARCHAR(100) NOT NULL,
    currency     VARCHAR(50)
);

CREATE TABLE exchange_rates (
    id INT AUTO_INCREMENT PRIMARY KEY,
    country_id INT NOT NULL,
    year INT NOT NULL,
    ama_exchange_rate DOUBLE,
    imf_based_exchange_rate DOUBLE,
    FOREIGN KEY (country_id) REFERENCES countries(country_id),
    UNIQUE KEY unique_country_year (country_id, year)
);

CREATE TABLE economic_indicators (
    id INT AUTO_INCREMENT PRIMARY KEY,
    country_id INT NOT NULL,
    year INT NOT NULL,
    population BIGINT,
    final_consumption_expenditure DOUBLE,
    gdp DOUBLE,
    -- 17 further indicator columns
    FOREIGN KEY (country_id) REFERENCES countries(country_id),
    UNIQUE KEY unique_country_year (country_id, year)
);

CREATE INDEX idx_economic_year ON economic_indicators(year);
CREATE INDEX idx_economic_country_year ON economic_indicators(country_id, year);
CREATE INDEX idx_exchange_year ON exchange_rates(year);
```

The loading script removes the padding spaces from the names and inserts the countries first, because the other two tables reference them, before inserting the exchange rates and the indicators. Missing values are stored as NULL rather than zero, so that they cannot be mistaken for genuine observations. The use of INSERT IGNORE together with the unique constraints prevents duplicate country-year rows if the script is run more than once.

Figures 12 and 13 show the output of the setup script when it is run against the MySQL server: the database and tables are created, 220 countries and 10,512 records are loaded, and the five queries described in Section 5.3 are executed.

```
PS C:\Users\josue\Music\ml-pipeline_spt\formative_1> python task2/mysql_setup.py
Creating database and tables...
Tables created!

Loading data from CSV...
Done! Loaded 220 countries and 10512 records.

============================================================
QUERY 1: Top 10 Countries by GDP (2021)
============================================================
  United States                 2021   GDP: 23,300,000,000,000
  China                         2021   GDP: 17,700,000,000,000
  Japan                         2021   GDP: 4,940,000,000,000
  Germany                       2021   GDP: 4,260,000,000,000
  India                         2021   GDP: 3,200,000,000,000
  United Kingdom                2021   GDP: 3,130,000,000,000
  France                        2021   GDP: 2,960,000,000,000
  Italy                         2021   GDP: 2,110,000,000,000
  Canada                        2021   GDP: 1,990,000,000,000
  Republic of Korea             2021   GDP: 1,810,000,000,000

============================================================
QUERY 2: US Economic Indicators (2015-2021)
============================================================
  United States  2015  GDP: 18,200,000,000,000  Exports: 2,270,000,000,000  Imports: 2,790,000,000,00
0
  United States  2016  GDP: 18,700,000,000,000  Exports: 2,230,000,000,000  Imports: 2,740,000,000,00
0
  United States  2017  GDP: 19,500,000,000,000  Exports: 2,390,000,000,000  Imports: 2,920,000,000,00
0
  United States  2018  GDP: 20,500,000,000,000  Exports: 2,540,000,000,000  Imports: 3,130,000,000,00
0
  United States  2019  GDP: 21,400,000,000,000  Exports: 2,540,000,000,000  Imports: 3,120,000,000,00
0
  United States  2020  GDP: 21,100,000,000,000  Exports: 2,150,000,000,000  Imports: 2,780,000,000,00
0
  United States  2021  GDP: 23,300,000,000,000  Exports: 2,540,000,000,000  Imports: 3,400,000,000,00
0
```

Figure 12. Output of python task2/mysql_setup.py: creating the tables, loading 220 countries and 10,512 records, and MySQL Queries 1 and 2.

```
QUERY 3: Average GDP by Currency (Top 10 in 2021)
============================================================
  Yuan Renminbi              Avg GDP: 17,700,000,000,000  Countries: 1
  Yen                        Avg GDP: 4,940,000,000,000  Countries: 1
  Indian Rupee               Avg GDP: 3,200,000,000,000  Countries: 1
  Pound Sterling             Avg GDP: 3,130,000,000,000  Countries: 1
  Canadian Dollar            Avg GDP: 1,990,000,000,000  Countries: 1
  US Dollar                  Avg GDP: 1,814,746,305,281  Countries: 13
  Won                        Avg GDP: 1,810,000,000,000  Countries: 1
  Russian Ruble              Avg GDP: 1,780,000,000,000  Countries: 1
  Brazilian Real             Avg GDP: 1,610,000,000,000  Countries: 1
  Mexican Peso               Avg GDP: 1,270,000,000,000  Countries: 1

============================================================
QUERY 4: Latest Record for United States
============================================================
  Country: United States
  Year: 2021
  GDP: 23,300,000,000,000
  Population: 336,997,624
  GNI per Capita: 70,081

============================================================
QUERY 5: GDP and Exchange Rates (US over decades)
============================================================
  1970  GDP: 1,070,000,000,000  Exchange Rate: 1.0
  1980  GDP: 2,860,000,000,000  Exchange Rate: 1.0
  1990  GDP: 5,960,000,000,000  Exchange Rate: 1.0
  2000  GDP: 10,300,000,000,000  Exchange Rate: 1.0
  2010  GDP: 15,000,000,000,000  Exchange Rate: 1.0
  2021  GDP: 23,300,000,000,000  Exchange Rate: 1.0

All MySQL queries completed.
```

Figure 13. Output of MySQL Queries 3 to 5: the average GDP by currency in 2021, the latest US record and US GDP with exchange rates by decade.

5.2 Document Design in MongoDB

MongoDB stores the same data in a single collection, economic_records, with one document per country and year (10,512 documents). Instead of distributing the data across tables, each document groups related fields into four embedded sub-documents: exchange_rates, demographics, economic_indicators and sector_breakdown. This design reflects the way in which the data are read: the API and the prediction script always retrieve complete country-year records, which in MongoDB requires a single document read and no joins. The country name and currency are repeated in each document; document databases accept this redundancy in exchange for simpler and faster reads. A unique compound index on (country_name, year) guarantees one document per country and year, in the same way as the unique constraint in MySQL, and together with an index on year it supports the same access patterns as the MySQL indexes. Listing 2 shows the document for the United States in 2021, as stored in the database.

Listing 2. Document for the United States in 2021 (the _id field is omitted).


```
{
  "country_id": 840,
  "country_name": "United States",
  "currency": "US Dollar",
  "year": 2021,
  "exchange_rates": {
    "ama_exchange_rate": 1.0,
    "imf_based_exchange_rate": 1.0
  },
  "demographics": {
    "population": 336997624,
    "per_capita_gni": 70081.0
  },
  "economic_indicators": {
    "gdp": 23300000000000.0,
    "gni_usd": 23600000000000.0,
    "exports": 2540000000000.0,
    "imports": 3400000000000.0,
    "final_consumption_expenditure": 19300000000000.0,
    "government_consumption": 3350000000000.0,
    "household_consumption": 15900000000000.0,
    "gross_capital_formation": 4920000000000.0,
    "gross_fixed_capital_formation": 4940000000000.0,
    "changes_in_inventories": -19091000000.0
  },
  "sector_breakdown": {
    "agriculture": 207000000000.0,
    "manufacturing": 2500000000000.0,
    "mining_manufacturing_utilities": 3270000000000.0,
    "construction": 945000000000.0,
    "transport_storage_communication": 2490000000000.0,
    "wholesale_retail": 3520000000000.0,
    "other_activities": 12900000000000.0,
    "total_value_added": 23300000000000.0
  }
}
```


Figure 14 shows the output of the setup script: the 10,512 documents are inserted, the indexes are created and the first sample document (Afghanistan, 1970) is printed.

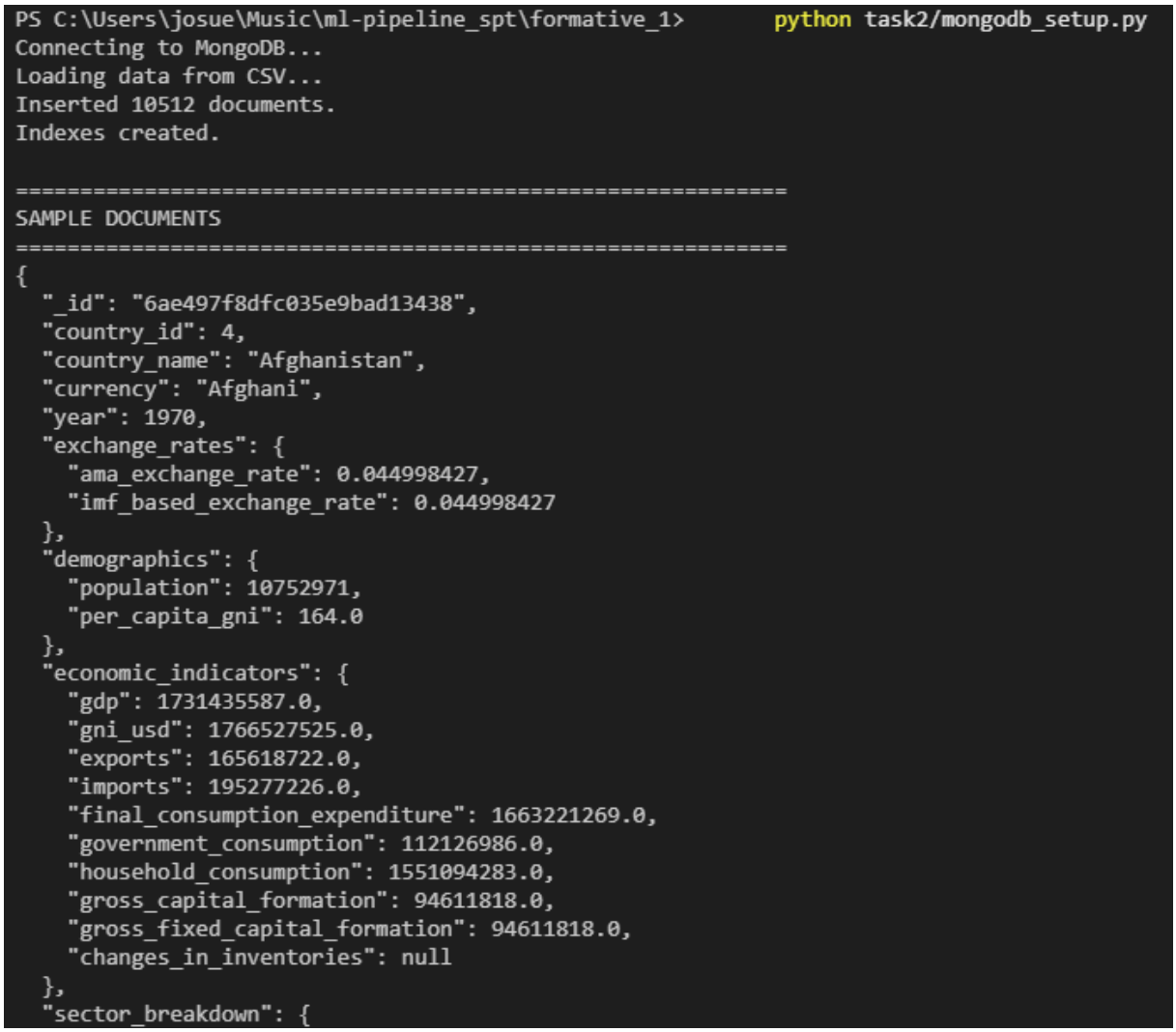


Figure 14. Output of python task2/mongodb_setup.py: loading the collection, creating the indexes and printing the first sample document.

5.3 Queries

Five queries were implemented in each database (Table 17). They cover ranking, filtering by a date range, aggregation, retrieval of the latest record and the combination of data from several sources. The statements of all ten queries and the results returned by the databases are given in Appendix A.

Table 17. Queries implemented in each database.

No.	MySQL	MongoDB
1	Ten largest economies in 2021 (JOIN, ORDER BY, LIMIT)	Ten largest economies in 2021 (find, sort, limit)
2	US indicators for 2015–2021 (WHERE ... BETWEEN)	US records for 2015–2021 (find with $gte and $lte)
3	Average GDP by currency in 2021 (GROUP BY, AVG)	Number of records per country ($group)

No.	MySQL	MongoDB
4	Latest US record (ORDER BY year DESC LIMIT 1)	Latest US record (find_one sorted by year, descending)
5	US GDP and exchange rates by decade (join of three tables)	Average GNI per capita by currency in 2021 ($match, $group)

Query 2 is the time-series query on which the date-range endpoint of the API is based. In MySQL it joins the indicators to the country names and filters the years with BETWEEN (Listing 3); in MongoDB the same filter is expressed with the $gte and $lte operators on the indexed year field (Listing 4). When executed, Query 1 returns the United States (US$23.3 trillion) and China (US$17.7 trillion) as the two largest economies in 2021, and Query 4 returns the 2021 record shown in Listing 2.

Listing 3. Query 2 in MySQL: US indicators for a range of years.

```
SELECT c.country_name, e.year, e.gdp, e.exports, e.imports
FROM economic_indicators e
JOIN countries c ON e.country_id = c.country_id
WHERE c.country_name = 'United States'
  AND e.year BETWEEN 2015 AND 2021
ORDER BY e.year;
```

Listing 4. Query 2 in MongoDB (Python, PyMongo).

```
collection.find(
    {"country_name": "United States", "year": {"$gte": 2015, "$lte": 2021}},
    {"_id": 0, "year": 1, "economic_indicators.gdp": 1,
     "economic_indicators.exports": 1, "demographics.population": 1},
).sort("year", 1)
```

Figures 15 and 16 show the output of the five MongoDB queries when the setup script is run.

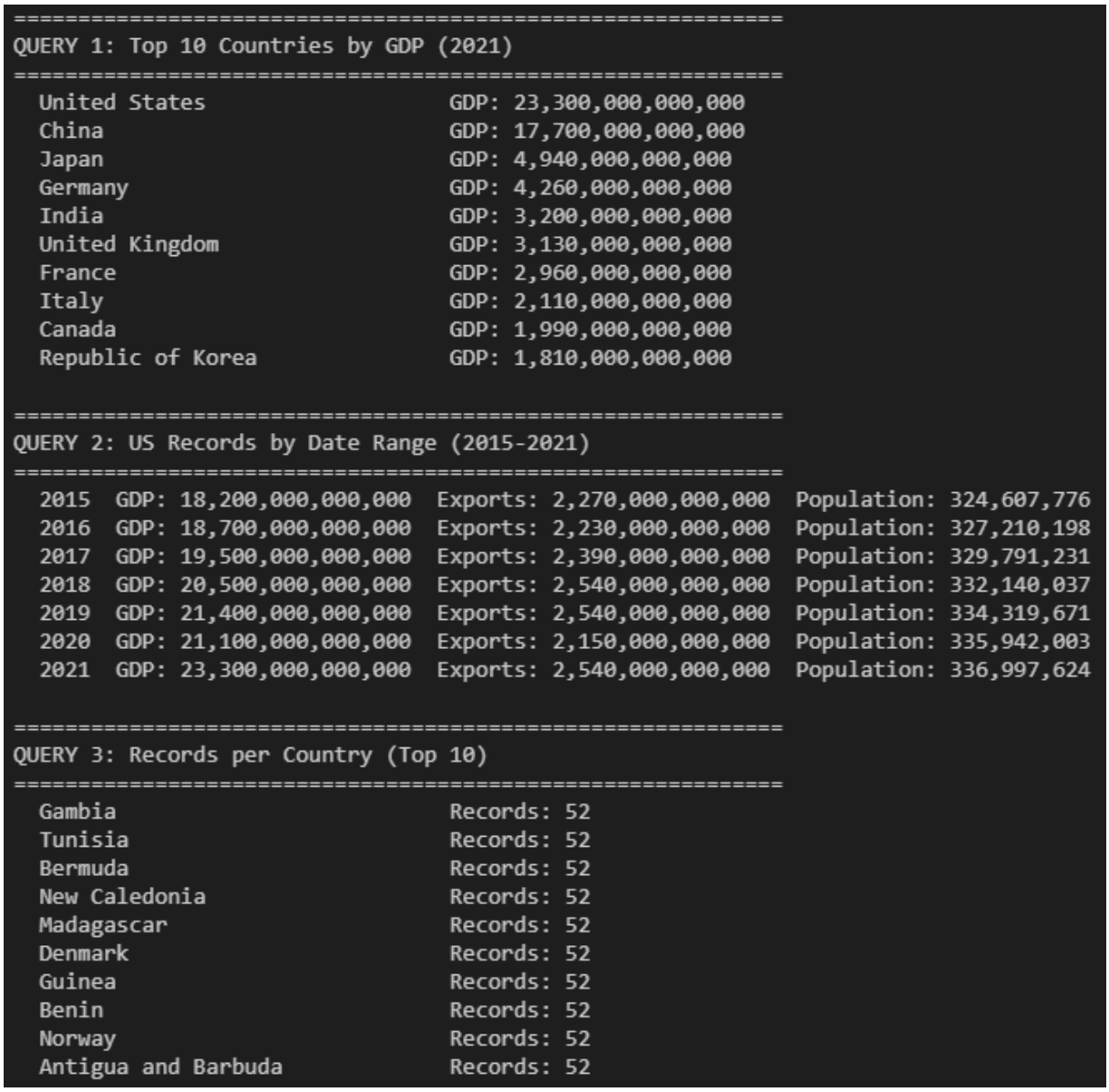


Figure 15. Output of MongoDB Queries 1 to 3: the ten largest economies in 2021, the US records for 2015–2021 and the number of records per country.

```
============================================================
QUERY 4: Latest Record for United States
============================================================
  Country: United States
  Year: 2021
  GDP: 23,300,000,000,000
  Population: 336,997,624
  GNI per Capita: 70,081

============================================================
QUERY 5: Avg GNI per Capita by Currency (Top 10, 2021)
============================================================
  Swiss Franc                Avg GNI/Capita: 144,170  Countries: 2
  Bermudian Dollar           Avg GNI/Capita: 112,607  Countries: 1
  Norwegian Krone            Avg GNI/Capita: 93,149  Countries: 1
  Iceland Krona              Avg GNI/Capita: 69,996  Countries: 1
  Qatari Rial                Avg GNI/Capita: 65,863  Countries: 1
  Danish Krone               Avg GNI/Capita: 63,502  Countries: 2
  Swedish Krona              Avg GNI/Capita: 62,469  Countries: 1
  Cayman Islands Dollar      Avg GNI/Capita: 61,982  Countries: 1
  Singapore Dollar           Avg GNI/Capita: 58,770  Countries: 1
  New Israeli Sheqel         Avg GNI/Capita: 53,302  Countries: 1

All MongoDB queries completed.
```

Figure 16. Output of MongoDB Queries 4 and 5: the latest US record and the average GNI per capita by currency in 2021.

5.4 Comparison of the Two Designs

Table 18 compares the two designs. The relational schema is better suited to enforcing consistency and to queries that combine data across tables, whereas the document model is better suited to reading complete records quickly.

Table 18. Comparison of the MySQL and MongoDB designs.

Aspect	**MySQL**	**MongoDB**
Structure	Three normalised tables linked by foreign keys	One collection of documents with embedded sub-documents
Reading one country-year record	Join of the three tables	Single document read
Data integrity	Enforced by primary keys, foreign keys and unique constraints	Unique index on (country_name, year); other rules enforced by the application
Changes to the structure	Require ALTER TABLE statements	New fields can be added to new documents
Most suitable for	Consistent related data and complex joins	Fast retrieval of complete records

6. API for CRUD Operations and Time-Series Queries

This section describes Task 3. The API is implemented with FastAPI in task3/api_demo.py. It exposes the same endpoints for both databases, under the prefixes /api/sql and /api/mongodb, so that a client can change databases by changing only the prefix. Request bodies are validated with Pydantic models: EconomicRecordCreate requires the country name and year and accepts all indicators as optional fields, while EconomicRecordUpdate contains only optional fields, which allows partial updates. FastAPI also generates interactive documentation at /docs. Table 19 lists the endpoints.

Table 19. Endpoints of the API (identical for /api/sql and /api/mongodb).

Method	**Path**	**Operation**	**MySQL implementation**	**MongoDB implementation**
POST	/records	Create a record	Inserts into economic_indicators (and exchange_rates when rates are given); adds the country if it does not exist; returns the identifier of the new indicator row	insert_one with the nested structure of Listing 2; a second record for the same country and year is rejected
GET	/records	Read all records of a country (parameter: country)	SELECT with joins, ordered by year	find, sorted by year
GET	/records/latest	Read the latest record of a country (parameter: country)	ORDER BY year DESC LIMIT 1	find_one, sorted by year in descending order
GET	/records/range	Read the records of every year that overlaps a date range (parameters: country, start_date, end_date)	year BETWEEN the start and end years	year with $gte and $lte
PUT	/records/{id}	Update a record	Checks that the record exists, then updates the supplied fields only, in the table that holds each field; exchange rates are inserted or updated	update_one with $set on the nested paths; matched_count and modified_count distinguish a missing record from an unchanged one
DELETE	/records/{id}	Delete a record	Deletes the indicator row and the matching exchange-rate row	delete_one by document identifier

Several design decisions apply to both databases. All read endpoints return the same flat JSON structure, including a Date field in YYYY-MM-DD format derived from the year, so the prediction script can process the output of either database without modification. Country names are matched without regard to letter case. In MySQL, inserts, updates and deletions that involve two tables are executed in one transaction and rolled back if any statement fails. Errors are reported with standard HTTP status codes: 400 for invalid input, such as an incomplete or non-existent date (for example 2015 or 2015-02-30), an end date before the start date or an update without fields; 404 when no

record matches; 409 when a record for the same country and year already exists; and 503 when the database cannot be reached. Dates are validated as complete calendar dates before they are converted into the years stored in the databases. Because the data are annual, each record represents a whole calendar year and carries the date of 1 January of that year; a date range therefore returns every year that overlaps it. A range from 2015-12-30 to 2015-12-31, for example, returns the 2015 record, shown with the date 2015-01-01, because the range lies within 2015.

Figure 17 shows the interactive documentation generated by FastAPI, which lists the endpoints and allows each of them to be tested from a web browser. Figures 18 and 19 show a request to the date-range endpoint of the MongoDB database, made through this documentation, and the response returned by the API.

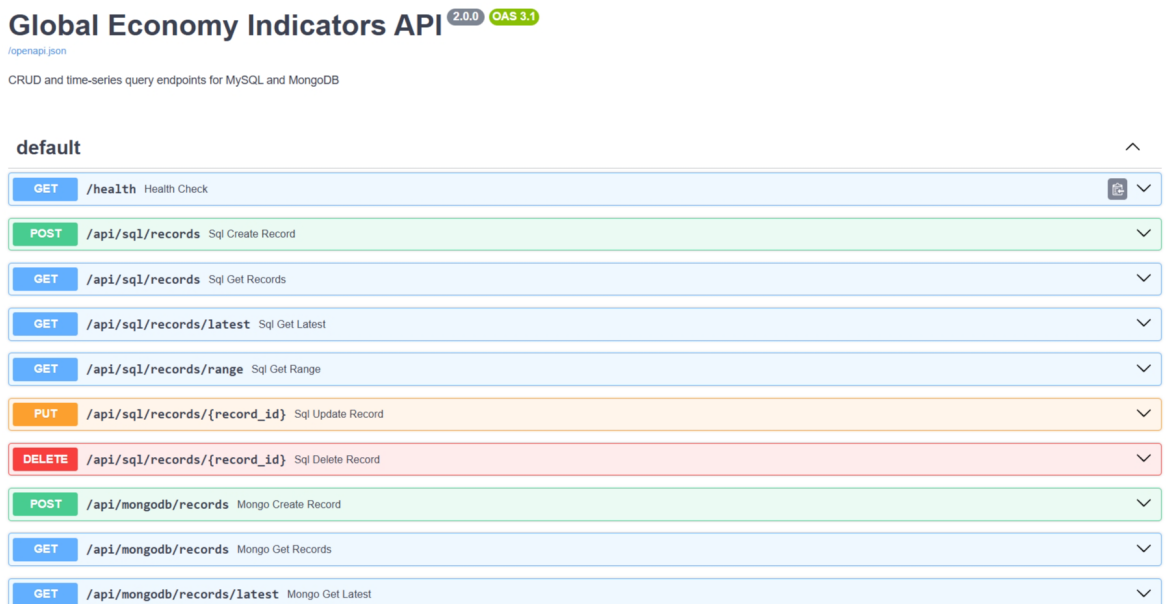


Figure 17. Interactive API documentation generated by FastAPI at /docs, showing the health check, the six MySQL endpoints and the first MongoDB endpoints; the remaining MongoDB endpoints follow on the same page.

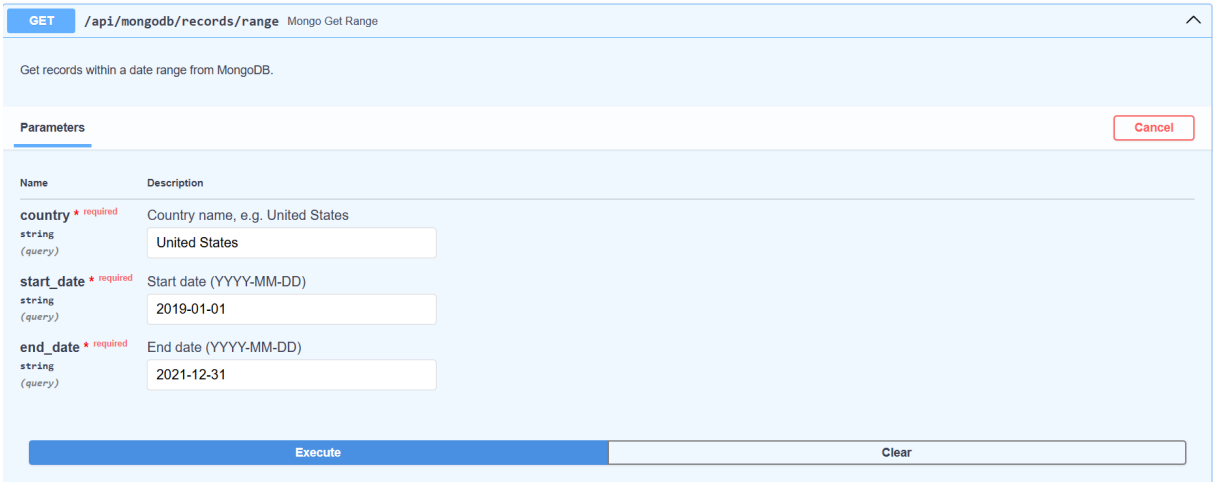


Figure 18. Request to GET /api/mongodb/records/range for the United States from 2019-01-01 to 2021-12-31, entered in the interactive documentation.

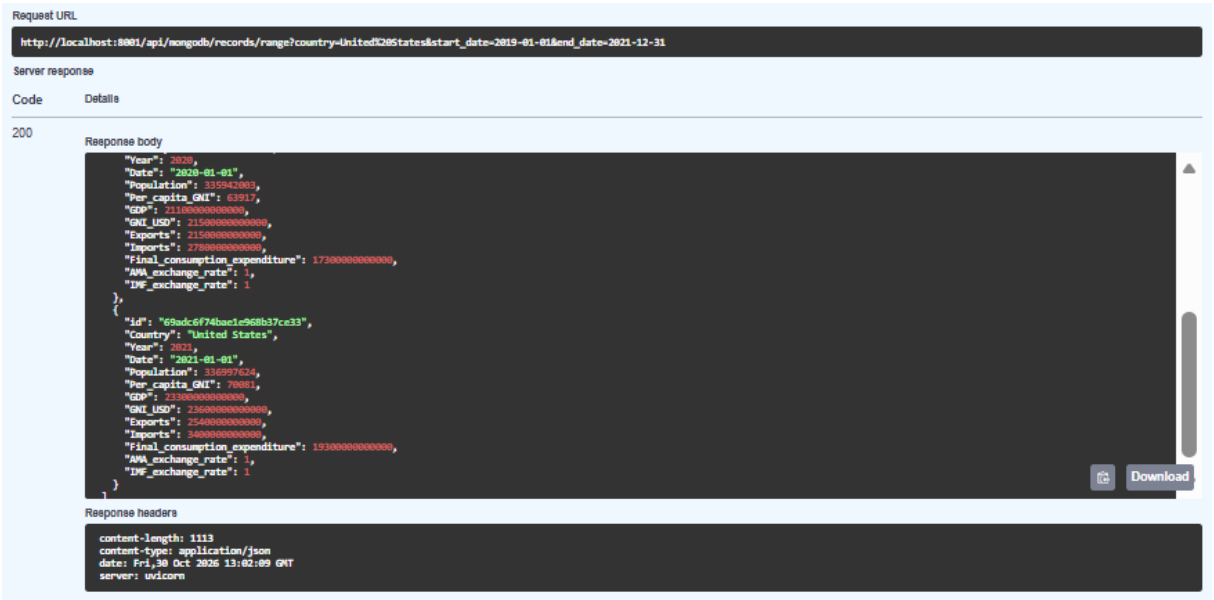


Figure 19. Response to the request in Figure 18: status code 200 and a response body with one record per year (the records for 2020 and 2021 are visible).

Listings 5 and 6 show the responses of the latest-record endpoint for each database, obtained from the running API, and Listing 7 shows two error responses.

Listing 5. Request for the latest US record from the MongoDB endpoints and the response.

```
GET /api/mongodb/records/latest?country=United%20States

HTTP 200
{
  "country": "United States",
  "data": {
    "id": "6ae4b23b96da56830ae4298d",
    "Country": "United States",
    "Year": 2021,
    "Date": "2021-01-01",
    "Population": 336997624,
    "Per_capita_GNI": 70081.0,
    "GDP": 23300000000000.0,
    "GNI_USD": 23600000000000.0,
    "Exports": 2540000000000.0,
    "Imports": 3400000000000.0,
    "Final_consumption_expenditure": 19300000000000.0,
    "AMA_exchange_rate": 1.0,
    "IMF_exchange_rate": 1.0
  }
}
```

Listing 6. Request for the latest US record from the MySQL endpoints and the response.

```
GET /api/sql/records/latest?country=United%20States

HTTP 200
{
  "country": "United States",
  "data": {
    "Country": "United States",
    "Year": 2021,
    "Population": 336997624,
    "Per_capita_GNI": 70081.0,
    "GDP": 23300000000000.0,
    "GNI_USD": 23600000000000.0,
    "Exports": 2540000000000.0,
    "Imports": 3400000000000.0,
    "Final_consumption_expenditure": 19300000000000.0,
    "AMA_exchange_rate": 1.0,
    "IMF_exchange_rate": 1.0,
    "record_id": 10145,
    "Date": "2021-01-01"
  }
}
```

The two responses contain the same values. They differ only in the identifier, a document identifier (id) in MongoDB and the integer key of the indicator row (record_id) in MySQL, and in the order of the fields.

Listing 7. Error responses for an invalid date range and an unknown country.

```
GET /api/mongodb/records/range?country=United%20States
    &start_date=2021-01-01&end_date=2015-12-31
HTTP 400   {"detail": "end_date must not be before start_date"}

GET /api/mongodb/records/latest?country=Atlantis
HTTP 404   {"detail": "No records found for country: Atlantis"}
```

6.1 Correctness of the Create and Update Operations

The first version of the API contained defects in the create and update operations, which were corrected. First, the MySQL create endpoint read the identifier of the new record only after the optional insert into exchange_rates, so when exchange rates were supplied it could return the identifier of the exchange-rate row instead of that of the indicator row; the identifier is now stored immediately after the indicator row is inserted. Second, the MySQL update endpoint interpreted zero affected rows as a missing record. MySQL, however, also reports zero affected rows when the new values equal the stored ones, so an update that changed nothing returned 404 for a record that exists. The endpoint now checks the existence of the record first and reports separately whether any value changed ("modified": 0 or 1); MongoDB makes the same distinction through matched_count and modified_count.

Third, an update of only the exchange rates skipped the existence check and reported success for a missing record, and it had no effect when the year had no exchange-rate row; it now returns 404 for a missing record and creates the exchange-rate row when none exists. In addition, the date-range endpoints previously accepted any value that pandas could convert into a date, including a bare year; they now require complete YYYY-MM-DD dates. Finally, a unique index on (country_name, year)

was added to the MongoDB collection, and a second record for an existing country and year is rejected with status 409 in both databases. Table 20 compares the responses of the two versions to the same requests.

Table 20. Responses of the first and the corrected version of the API to the same MySQL requests.

Request	First version	Corrected version
PUT /records/10145 with the GDP value already stored (United States, 2021)	404, record not found	200, "modified": 0
PUT /records/999999 with only an exchange rate (no such record)	200, "Record updated"	404, record not found
GET /records/range with start_date 2015 and end_date 2016	200, records for 2015–2016	400, invalid start_date

6.2 Demonstration of the CRUD Workflow

The script task3/crud_demo.py exercises the complete workflow against the running API for both databases and checks every response. It creates a record for the United States in 2030, a year that is not in the dataset, reads it back, updates it, reads it again, repeats the same update, deletes the record and confirms that it can no longer be read. It then requests a date range within a single year, which returns the record for that whole year, and sends the invalid requests listed in the lower part of Table 21. All 45 checks passed; the complete output is stored in report/crud_demo_output.txt, and Listing 8 shows an excerpt for MySQL.

Table 21. Results of the CRUD demonstration (status code and main content of each response).

Step	Request	MySQL	MongoDB
1	POST /records: United States, 2030	201, new id returned	201, new id returned
2	GET /records/range for 2030	200, same id and GDP as sent	200, same id and GDP as sent
3	PUT /records/{id}: new GDP and exchange rates	200, modified 1	200, modified 1
4	GET /records/range for 2030	200, new values stored	200, new values stored
5	PUT /records/{id} with the same values	200, modified 0	200, modified 0
6	DELETE /records/{id}	200, record deleted	200, record deleted
7	GET /records/range for 2030	404, deletion confirmed	404, deletion confirmed
A1	GET /records/range from 2015-12-30 to 2015-12-31	200, the 2015 record	200, the 2015 record
E1	PUT with a nonexistent id	404	404
E2	PUT of an exchange rate only, nonexistent id	404	404
E3	DELETE with a nonexistent id	404	404
E4	GET /records/range with start_date 2015	400	400

Step	Request	MySQL	MongoDB
E5	GET /records/range with start_date 2015-02-30	400	400
E6	GET /records/range ending before it starts	400	400
E7	POST a second record for United States, 2021	409	409
E8	PUT with an id that is not a valid MongoDB id	Not applicable	400

Listing 8. Excerpt from the output of python task3/crud_demo.py for MySQL (request bodies and read responses omitted).

```
[MySQL 1] Create a record for United States 2030
  POST /api/sql/records
  -> 201 (expected 201)  PASS
  {"message": "Record created", "id": 10519, "country": "United States", "year": 2030}

[MySQL 3] Update GDP and add exchange rates (the record has none yet)
  PUT /api/sql/records/10519
  -> 200 (expected 200)  PASS
  {"message": "Record updated", "id": 10519, "modified": 1}

[MySQL 5] Send the same update again (changes nothing, the record still exists)
  PUT /api/sql/records/10519
  -> 200 (expected 200)  PASS
  {"message": "No changes: the record already has these values", "id": 10519, "modified": 0}

[MySQL 6] Delete the record
  DELETE /api/sql/records/10519
  -> 200 (expected 200)  PASS

[MySQL 7] Read it again: the record is gone
  GET /api/sql/records/range?country=United States&start_date=2030-01-01&end_date=2030-12-31
  -> 404 (expected 404)  PASS
  {"detail": "No records in the specified date range"}
```

7. Prediction Script

This section describes Task 4. The prediction script, task4/predict_from_api.py, connects the components described above and performs the four steps listed in Table 22.

Table 22. Steps of the prediction script.

Step	Operation	Description
1	Fetch	Reads preprocess_config.json and refuses any country other than the one on which the model was trained; then requests the records of the country from the API: all records, or a date range through /records/range when start and end dates are given. MongoDB is used by default and MySQL with the option --db sql
2	Preprocess	Converts the JSON response into a table indexed by year and applies the Task 1 preprocessing to the years up to the one being estimated: one row per year, interpolation of missing values, GDP_Lag1 and GDP_5Year_MA with the window read from preprocess_config.json

Step	Operation	Description
3	Load the model	Loads the Linear Regression model and the feature order saved by task4/train_model.py, checks that they match each other and the preprocessing code, and warns if the installed scikit-learn version differs from the one used for training
4	Predict	Estimates GDP for the latest year, or for the year given with --year, and compares the estimate with the actual value, where available, and with the naive baseline

7.1 Consistency with Task 1

To ensure that the model receives exactly the same inputs as in Task 1, the cleaning and feature-engineering steps are implemented once, in task4/preprocessing.py, and used by both the training script and the prediction script. The training script, task4/train_model.py, trains the model on the same data, features and chronological split as the notebook and reproduces its test results (RMSE US$889 billion, MAPE 4.17%, R^2 0.82). It saves the model with joblib, together with the feature order, the preprocessing configuration and a summary of the training run. GDP is interpolated only between known years, so a year without a GDP value, for example a new record created through the API, remains empty and can be predicted.

The settings saved with the model are enforced rather than assumed. Before fetching any data, the script reads preprocess_config.json and stops if the requested country differs from the country on which the model was trained. The model describes the relationship between the features and GDP for the United States only, and applying it to another economy would produce estimates without a validated basis; another country can be used only by supplying, with --artifacts_dir, a model trained for that country. The moving-average window is read from the same file, and the target, the lag and the feature order are checked against the preprocessing code, so that the features are constructed exactly as during training even if the code is later changed.

For historical estimates, the preprocessing uses only the information that would have been available at the time. Before missing values are filled, the records are truncated at the year being estimated and the GDP value of that year is removed. Without this step, a missing GDP value for the preceding year would be interpolated between its neighbours, one of which is the GDP being estimated, so that the target would leak into its own GDP_Lag1 feature; with it, such a gap cannot be filled and the script reports that the history is insufficient. The change does not affect the results reported here, because the modelled US columns contain no missing values.

7.2 Example Run

Listing 9 shows the output of the script when it requests the US records for 2010–2021 from the running API through the date-range endpoint.

Listing 9. Output of python task4/predict_from_api.py --start_date 2010-01-01 --end_date 2021-12-31.

```
[1/4] Fetching records for United States from http://localhost:8000/api/mongodb/records/range
      12 records received
[2/4] Preprocessing (same pipeline as Task 1: interpolation, GDP_Lag1, GDP_5Year_MA)
      Years 2010-2021, one row per year (moving-average window 5 from preprocess_config.json)
[3/4] Loading the trained model
      Features: ['Population', 'GDP_Lag1', 'GDP_5Year_MA', 'Final_consumption_expenditure']
[4/4] Making the prediction

=== GDP estimate: United States, 2021 ===
Model: LinearRegression trained on 1975-2011 (test R2 0.824, MAPE 4.17%)
Inputs:
  Population                                        336,997,624
  GDP_Lag1 (GDP 2020)                          US$21.10 trillion
  GDP_5Year_MA (mean GDP 2016-2020)            US$20.24 trillion
  Final_consumption_expenditure                US$19.30 trillion

Predicted GDP 2021: US$22.14 trillion
Actual GDP 2021 (from API): US$23.30 trillion -> model error -5.0%
Naive baseline (GDP 2020): US$21.10 trillion -> baseline error -9.4%
```

The model estimates 2021 GDP at US$22.14 trillion against an actual value of US$23.30 trillion, an error of −5.0%, whereas the naive baseline has an error of −9.4%. Both values are identical to the 2021 column of Table 14, which confirms that the path from the API through preprocessing to the model reproduces the results of Task 1. Figure 20 shows a second run, in which the script fetched all 52 US records through the /records endpoint and produced the same estimate.

```
PS C:\Users\josue\Music\ml-pipeline_spt\formative_1> python task4/predict_from
_api.py --api http://localhost:8001
[1/4] Fetching records for United States from http://localhost:8001/api/mongod
b/records
      52 records received
[2/4] Preprocessing (same pipeline as Task 1: interpolation, GDP_Lag1, GDP_5Ye
ar_MA)
      Years 1970-2021, one row per year
[3/4] Loading the trained model
      Features: ['Population', 'GDP_Lag1', 'GDP_5Year_MA', 'Final_consumption_
expenditure']
[4/4] Making the prediction

=== GDP estimate: United States, 2021 ===
Model: LinearRegression trained on 1975-2011 (test R2 0.824, MAPE 4.17%)
Inputs:
  Population                                        336,997,624
  GDP_Lag1 (GDP 2020)                          US$21.10 trillion
  GDP_5Year_MA (mean GDP 2016-2020)            US$20.24 trillion
  Final_consumption_expenditure                US$19.30 trillion

Predicted GDP 2021: US$22.14 trillion
Actual GDP 2021 (from API): US$23.30 trillion -> model error -5.0%
Naive baseline (GDP 2020): US$21.10 trillion -> baseline error -9.4%
```

Figure 20. Terminal output of python task4/predict_from_api.py --api http://localhost:8001. The script fetched all 52 US records (1970–2021) and estimated 2021 GDP at US$22.14 trillion, an error of −5.0% against −9.4% for the naive baseline.

7.3 Error Handling

The script stops with an explanatory message and exit code 1 when the API is unavailable or returns an error (for example, for an unknown country or an invalid date), when only one of the two dates is supplied, when the requested country is not the one on which the model was trained, when fewer than six consecutive years are available (the five-year moving average requires five previous years), when the requested year is not among the records, when the model has not been trained, or when the model, its feature order and its configuration do not match each other or the preprocessing code. If the population or consumption value for the year being estimated is missing, it is filled from earlier years and a warning is printed.

7.4 Reproducibility

The results in this report can be reproduced from the repository. The package versions with which the code was tested are recorded in requirements.txt (Python 3.13.9, pandas 2.3.3, NumPy 2.3.5 and scikit-learn 1.7.2, among others), and the training script saves the Python, scikit-learn, pandas and NumPy versions in training_summary.json. scikit-learn is pinned to version 1.7.2, because a model stored with joblib is a serialised Python object that may not load identically under another version; the prediction script warns when the installed version differs. The random elements of the analysis are fixed: the tree-based models use random_state=42, and the bootstrapped confidence band in Figure 7 uses a fixed seed, so that repeated runs of the notebook produce identical results and figures. The databases were tested with MySQL 8.4.8 and MongoDB 8.2.5.

8. Conclusion

This project implemented a complete pipeline for annual time-series data, from exploratory analysis to a prediction service. The analysis showed that US GDP follows a strong trend, for which the ADF test fails to reject a unit root, that the previous year's GDP and same-year consumption carry most of the predictive information, and that population correlates with GDP mainly through the shared trend. Linear Regression with all four features was selected on time-ordered validation folds within the training period and, when evaluated on the test period after this selection, outperformed the naive baseline (RMSE US$889 billion compared with US$968 billion), because it can extrapolate beyond the training range, whereas the tree-based models cannot. The data are stored in a normalised MySQL schema and in a MongoDB collection of embedded documents, served through a FastAPI interface with identical endpoints for both databases, whose complete CRUD workflow was demonstrated on both databases, and used by a prediction script that enforces the settings of the trained model and reproduces the results of Task 1 exactly.

The main limitations are the small number of annual observations, the use of nominal values, which include inflation, the evaluation on final revised values rather than the preliminary figures available in real time, the fact that the test period was examined during earlier development and is therefore not fully independent, and the increasing under-prediction in the later test years. Future work could retrain the model each year with walk-forward validation, use inflation-adjusted GDP, extend the analysis to other countries and add authentication to the API before it is made available beyond a local environment.

9. Team Contributions

The work was shared between the two members as shown in Table 23.

Table 23. Contributions of the team members.

Member	Tasks	Main components
Josue Byiringiro	Tasks 1 and 4	Problem definition and dataset justification; exploratory analysis, preprocessing and feature engineering; model and feature selection and evaluation; shared preprocessing module, model training script and prediction script; saved model and its configuration; report and README
Celine Shoga	Tasks 2 and 3	Project requirements and settings template; relational schema and entity relationship diagram in MySQL; document design in MongoDB; setup scripts that load the data and run the five queries in each database; unique country-year index in MongoDB; FastAPI service with the CRUD, latest-record and date-range endpoints for both databases, with request validation and error handling; script that demonstrates the complete CRUD workflow on both databases; report and README

References

Dickey, D. A., & Fuller, W. A. (1979). Distribution of the estimators for autoregressive time series with a unit root. Journal of the American Statistical Association, 74(366a), 427–431.

European Commission, International Monetary Fund, Organisation for Economic Co-operation and Development, United Nations, & World Bank. (2009). System of National Accounts 2008. United Nations.

Hyndman, R. J., & Athanasopoulos, G. (2021). Forecasting: Principles and practice (3rd ed.). OTexts. https://otexts.com/fpp3/

Pedregosa, F., Varoquaux, G., Gramfort, A., Michel, V., Thirion, B., Grisel, O., Blondel, M., Prettenhofer, P., Weiss, R., Dubourg, V., Vanderplas, J., Passos, A., Cournapeau, D., Brucher, M., Perrot, M., & Duchesnay, É. (2011). Scikit-learn: Machine learning in Python. Journal of Machine Learning Research, 12, 2825–2830.

prasad22. (n.d.). Global economy indicators [Data set]. Kaggle. https://www.kaggle.com/datasets/prasad22/global-economy-indicators

Seabold, S., & Perktold, J. (2010). Statsmodels: Econometric and statistical modeling with Python. Proceedings of the 9th Python in Science Conference.

U.S. Bureau of Economic Analysis. (n.d.-a). Gross domestic product. https://www.bea.gov/data/gdp/gross-domestic-product

U.S. Bureau of Economic Analysis. (n.d.-b). Personal income and outlays. https://www.bea.gov/data/income-saving/personal-income

U.S. Census Bureau. (n.d.). Population estimates: Release schedule. https://www.census.gov/programs-surveys/popest/about/schedule.html

United Nations Statistics Division. (n.d.). National Accounts Main Aggregates Database. https://unstats.un.org/unsd/snaama/

Appendix A. Database Queries and Results

This appendix gives the five queries implemented in each database, as executed by task2/mysql_setup.py and task2/mongodb_setup.py (only the layout of the statements has been adjusted), together with the results returned by the databases. Monetary values are rounded to two decimals in US$ trillion or to whole US$ billion, as stated in each table. Query 2 of each database, the date-range query, is shown in Listings 3 and 4 in Section 5.3.

A.1 MySQL

Listing A1. MySQL Query 1: the ten largest economies in the latest year.

```
SELECT c.country_name, e.year, e.gdp
FROM economic_indicators e
JOIN countries c ON e.country_id = c.country_id
WHERE e.year = (SELECT MAX(year) FROM economic_indicators)
  AND e.gdp IS NOT NULL
ORDER BY e.gdp DESC
LIMIT 10;
```

Table A1. Result of MySQL Query 1 (year 2021).

Rank	Country	GDP (US$ trillion)
1	United States	23.30
2	China	17.70
3	Japan	4.94
4	Germany	4.26
5	India	3.20
6	United Kingdom	3.13
7	France	2.96
8	Italy	2.11
9	Canada	1.99
10	Republic of Korea	1.81

Table A2. Result of MySQL Query 2 (Listing 3): US indicators for 2015–2021.

Year	GDP (US$ trillion)	Exports (US$ trillion)	Imports (US$ trillion)
2015	18.20	2.27	2.79
2016	18.70	2.23	2.74
2017	19.50	2.39	2.92
2018	20.50	2.54	3.13
2019	21.40	2.54	3.12
2020	21.10	2.15	2.78

Year	GDP (US$ trillion)	Exports (US$ trillion)	Imports (US$ trillion)
2021	23.30	2.54	3.40

Listing A2. MySQL Query 3: average GDP by currency in 2021.

```
SELECT c.currency, ROUND(AVG(e.gdp), 0) AS avg_gdp, COUNT(*) AS num_countries
FROM economic_indicators e
JOIN countries c ON e.country_id = c.country_id
WHERE e.year = 2021 AND e.gdp IS NOT NULL
GROUP BY c.currency
ORDER BY avg_gdp DESC
LIMIT 10;
```

Table A3. Result of MySQL Query 3.

Currency	Average GDP (US$ billion)	Countries
Yuan Renminbi	17,700	1
Yen	4,940	1
Indian Rupee	3,200	1
Pound Sterling	3,130	1
Canadian Dollar	1,990	1
US Dollar	1,815	13
Won	1,810	1
Russian Ruble	1,780	1
Brazilian Real	1,610	1
Mexican Peso	1,270	1

Listing A3. MySQL Query 4: the latest record for the United States.

```
SELECT c.country_name, e.year, e.gdp, e.population, e.per_capita_gni
FROM economic_indicators e
JOIN countries c ON e.country_id = c.country_id
WHERE c.country_name = 'United States'
ORDER BY e.year DESC
LIMIT 1;
```

Table A4. Result of MySQL Query 4.

Country	Year	GDP (US$ trillion)	Population	GNI per capita (US$)
United States	2021	23.30	336,997,624	70,081

Listing A4. MySQL Query 5: US GDP and exchange rates by decade (join of the three tables).

```
SELECT c.country_name, e.year, e.gdp, er.ama_exchange_rate
FROM economic_indicators e
JOIN countries c ON e.country_id = c.country_id
JOIN exchange_rates er ON e.country_id = er.country_id AND e.year = er.year
WHERE c.country_name = 'United States'
  AND e.year IN (1970, 1980, 1990, 2000, 2010, 2021)
ORDER BY e.year;
```

Table A5. Result of MySQL Query 5.

Year	GDP (US$ trillion)	AMA exchange rate
1970	1.07	1.0
1980	2.86	1.0
1990	5.96	1.0
2000	10.30	1.0
2010	15.00	1.0
2021	23.30	1.0

A.2 MongoDB

Listing A5. MongoDB Query 1 (Python, PyMongo): the ten largest economies in 2021.

```
collection.find(
    {"year": 2021, "economic_indicators.gdp": {"$ne": None}},
    {"_id": 0, "country_name": 1, "economic_indicators.gdp": 1},
).sort("economic_indicators.gdp", -1).limit(10)
```

Table A6. Result of MongoDB Query 1.

Rank	Country	GDP (US$ trillion)
1	United States	23.30
2	China	17.70
3	Japan	4.94
4	Germany	4.26
5	India	3.20
6	United Kingdom	3.13
7	France	2.96
8	Italy	2.11
9	Canada	1.99
10	Republic of Korea	1.81

Table A7. Result of MongoDB Query 2 (Listing 4): US records for 2015–2021.

Year	GDP (US$ trillion)	Exports (US$ trillion)	Population
2015	18.20	2.27	324,607,776
2016	18.70	2.23	327,210,198
2017	19.50	2.39	329,791,231
2018	20.50	2.54	332,140,037
2019	21.40	2.54	334,319,671
2020	21.10	2.15	335,942,003
2021	23.30	2.54	336,997,624

Listing A6. MongoDB Query 3: number of records per country.

```
collection.aggregate([
    {"$group": {"_id": "$country_name", "record_count": {"$sum": 1}}},
    {"$sort": {"record_count": -1}},
    {"$limit": 10},
])
```

Table A8. Result of MongoDB Query 3. All ten countries have the maximum of 52 records; because 179 countries share this count, which ten of them are returned is not determined by the sort.

Country	Records
India	52
Liberia	52
Turks and Caicos Islands	52
United Arab Emirates	52
Somalia	52
Maldives	52
Luxembourg	52
Lao People's DR	52
Cayman Islands	52
Botswana	52

Listing A7. MongoDB Query 4: the latest record for the United States.

```
collection.find_one(
    {"country_name": "United States"},
    {"_id": 0},
    sort=[("year", -1)],
)
```

Table A9. Result of MongoDB Query 4 (selected fields; the complete document is shown in Listing 2).

Country	Year	GDP (US$ trillion)	Population	GNI per capita (US$)
United States	2021	23.30	336,997,624	70,081

Listing A8. MongoDB Query 5: average GNI per capita by currency in 2021.

```
collection.aggregate([
    {"$match": {"year": 2021, "demographics.per_capita_gni": {"$ne": None}}},
    {"$group": {
        "_id": "$currency",
        "avg_gni": {"$avg": "$demographics.per_capita_gni"},
        "num_countries": {"$sum": 1},
    }},
    {"$sort": {"avg_gni": -1}},
    {"$limit": 10},
])
```

Table A10. Result of MongoDB Query 5.

Currency	Average GNI per capita (US$)	Countries
Swiss Franc	144,170	2
Bermudian Dollar	112,607	1
Norwegian Krone	93,149	1
Iceland Krona	69,996	1
Qatari Rial	65,863	1
Danish Krone	63,502	2
Swedish Krona	62,469	1
Cayman Islands Dollar	61,982	1
Singapore Dollar	58,770	1
New Israeli Sheqel	53,302	1